

LISTA DE ILUSTRAÇÕES

Figura 1 – Estrutura da transformação e validação do contrato	7
Figura 2 – Estrutura de análise do solidClarva com contractLarva	8
Figura 3 – Elementos de um Rede de Petri.	25
Figura 4 – Rede de Petri de uma máquina de lavar.	26
Figura 5 – Rede de Petri colorida para um restaurante.	31
Figura 6 – Rede de Petri Colorida R_{lit} gerada a partir do contrato inteligente . . .	51
Figura 7 – Contrato de compra e venda em RCL	58
Figura 8 – Contrato de compra e venda corrigido em RCL	60

LISTA DE TABELAS

Tabela 1	–	Tabela <i>Variáveis Globais</i> extraída do contrato Restaurante.	47
Tabela 2	–	Tabela <i>Funções</i> extraída do contrato Restaurante.	47
Tabela 3	–	Tabela <i>Operações</i> extraída do contrato Restaurante.	48
Tabela 4	–	Tabela <i>Condicionais</i> extraída do contrato Restaurante.	48
Tabela 5	–	Conjunto de lugares gerados para o contrato restaurante.	49
Tabela 6	–	Comparativo entre a RPC da Figura 5 e a RPC obtida.	52
Tabela 7	–	Ações do contrato	57

LISTA DE ABREVIATURAS E SIGLAS

SC	Smart Contract
----	----------------

SUMÁRIO

1	INTRODUÇÃO	6
1.1	Verificação de smart contracts!?	6
1.2	Azzopardi!?	7
1.3	Verificação de contratos eletrônicos	9
2	FUNDAMENTAÇÃO TEÓRICA	11
2.1	Contratos	11
2.1.1	Contratos Eletrônicos	12
2.2	Verificação de Contratos	13
2.3	Blockchain	15
2.3.1	Blocos e Transações	16
2.3.2	Smart Contracts	17
2.4	Smart Contracts	18
2.4.1	Aspectos Legais	18
2.4.2	Execução dos Smart Contracts	19
2.5	Blockchain de Propósito Geral	20
2.5.1	Ethereum	20
2.5.2	Solidity	21
2.5.3	Remix IDE	24
2.6	Redes de Petri	24
2.6.1	Redes de Petri Ordinárias	25
2.6.2	Redes de Petri coloridas	28
3	DE CONTRATOS INTELIGENTES PARA REDES DE PETRI	35
3.1	Uma Abordagem de Transformação	35
3.2	Arquitetura da Transformação	37
3.3	Algoritmo de Transformação	39
3.4	Aplicação do Algoritmo	45
3.4.1	Análise o Contrato Solidity	46
3.4.2	Obtendo a Rede de Petri Colorida	48
3.4.3	A RPC obtida para o Restaurante	51
4	APLICAÇÃO PRÁTICA	56
4.1	Um Contrato em RCL	56
4.2	Um Contrato Inteligente em Solidity	60
4.2.1	Implementação em Solidity	60

4.2.1.1	Versão Corrigida em Solidity	67
5	CONCLUSÃO	71
	REFERÊNCIAS	72

1 INTRODUÇÃO

ALB: Aqui vem a introducao do trabalho

ALB: Em seguida, a ideia é estabelecer a conexao com trabalhos existentes, na mesma linha e/ou que tem relacao com o trabalho

A verificação de *smart contracts* é um assunto que já é discutido por diversos pesquisadores, algumas abordagens diferentes das utilizadas nesse trabalho já foram propostas.

1.1 Verificação de smart contracts!?

[1] propuseram um sistema de verificação de contratos escritos em solidity usando redes de Petri coloridas. O fluxo de verificação proposto consiste na tradução do contrato para redes de Petri usando a ferramenta ANTLR, para transformar o código em uma *Abstract Syntax Tree*, uma estrutura que será por fim, transformada em uma rede de Petri para que a verificação possa ser feita. O autor também engloba a transformação de bytecode de um contrato na *Ethereum Virtual Machine* para uma rede de Petri.

Para o código-fonte, a metodologia envolve a geração de uma Árvore de Sintaxe Abstrata (Abstract Syntax Tree - AST) para extrair a lógica de negócio e convertê-la em um modelo de Redes de Petri coloridas (RPC). A principal vantagem de utilizar uma AST é que ela fornece uma representação formal, hierárquica e sem ambiguidades de todo o código-fonte, capturando todas as suas estruturas, como declarações de variáveis, funções e fluxos de controle. A transformação ocorre, de fato, pela aplicação de um conjunto de regras de mapeamento sobre essa árvore.

Para o bytecode, o processo consiste em construir um Grafo de Fluxo de Controle (Control Flow Graph - CFG) a partir de blocos básicos de instruções, que é então usado para gerar um modelo CPN que simula a execução dinâmica na EVM. O trabalho também melhora as regras de lógica de programa existentes, aplicando uma versão simplificada da lógica de Hoare para definir pré e pós-condições para as operações em nível de instrução e de bloco básico.

Um diferencial da proposta é a introdução de uma estratégia de derivação de caminho baseada em backtracking para otimizar a análise. Essa técnica utiliza uma biblioteca de vulnerabilidades conhecidas para definir “estados-alvo” e deriva os caminhos que podem alcançá-los de forma reversa. Isso permite simplificar o modelo CPN ao reduzir o espaço de estados a ser explorado, melhorando a eficiência da simulação e da verificação (model checking). O fluxo de transformação de verificação é descrita na imagem a seguir.

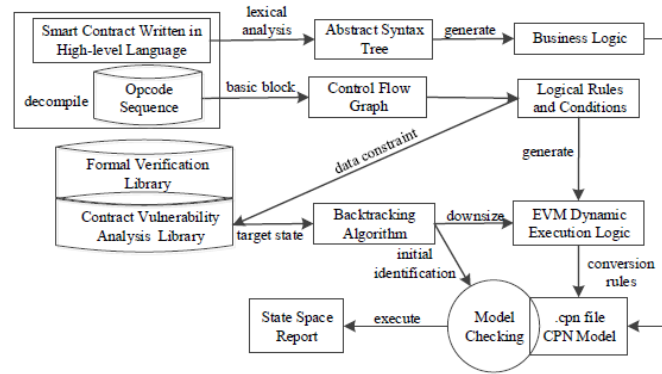


Figura 1 – Estrutura da transformação e validação do contrato

Embora a abordagem de [1] seja parecida com a abordagem adotada por esse trabalho, a verificação do contrato é feita através do próprio modelo RPC, pela aplicação CPN Tools. A ferramenta executa uma simulação dinâmica do modelo para explorar todo o seu espaço de estado. Isso permite encontrar caminhos de execução que levam a erros ou a estados indesejados, além de gerar relatórios do espaço de estados (State Space Report) que ajudam a identificar vulnerabilidades como reentrancy, problemas de concorrência e falhas na lógica de negócio.

ALB: A ver esse ultimo, pegar falhas na logica de negocio.

O artigo também não descreve em detalhes como é a transformação do modelo AST para RPC, o que dificulta a aplicação desse método. ALB: Ok, esse é um ponto importante, e que deve ser abordado com cuidado: dizer qual é a fraqueza/falha/problema da abordagem.

ALB: A seguir já é outro trabalho? Entao colocariamos outras secao, correto?

1.2 Azzopardi!?

[2] propôs uma abordagem diferente para o problema da verificação. A abordagem usada foi a da utilização de autômatos, nesse trabalho são utilizados dois tipos de autômatos para fazer a verificação de um contrato. Os *Control Flow Automata* (CFAs) são utilizados para descrever as rotinas de um contrato, com lógica passo a passo do programa codificada em rótulos de transição e chamadas de função por meio de rótulos de estado.

ALB: Essa parte é pega por um automato (fluxo), assim como queremos pela analise do cogido e estamos tenando isso com um diagrama de classes (uma representacao estatica), mas que da uma expressividade distinta. Vamos precisar de um modelo extra e/ou um algoritmo (sistematico) que daranta a conversao do codigo? Os *Dynamic Event Automata* definem os comportamentos do contrato, esses autômatos são usados para verificar se um contrato obedece certas especificações. ALB: Essa parte é interessante, e como ele faz isso? Essa semantica (comportamento) pega o que a RPC (do trabalho anterior e como

queremos) tambem pega?

A junção desses dois autômatos consegue monitorar o contrato para que esse obtenha o menor número de erros possíveis. Combinando análise estática e verificação em tempo de execução (runtime verification). A principal motivação é superar as limitações das abordagens puras: a análise estática pode ser excessivamente cara ou inconclusiva para contratos complexos, enquanto a verificação em tempo de execução, embora completa para um traço de execução, impõe custos de gas adicionais para a instrumentação e monitoramento do contrato.

ALB: Parece interessante. E como fazem isso? Tem a ferramenta/implementacao e/ou a aplicacao? Teria como replicar o estudo pratico que estamos fazendo nele para fins de comparacao?

MSN: respondendo os 2 comentários anteriores: Baseado no contrato em solidity e no DEA, o solidClarva faz a análise para identificar o que precisa ser verificado em runtime e a saída é o DEA residual, sem as propriedades que não precisam ser verificadas em runtime. Porém, ele não gera o smart contract residual. Aí o contractLarva recebe o DEA residual e o contrato original, após rodar o contractLarva o resultado é um novo contrato monitorado.

O ponto central da abordagem é a “análise residual” (residual analysis) ou verificação parcial. Uma análise estática inicial é realizada compondo o CFA do contrato com o DEA da propriedade. Essa análise, que é sólida, mas não necessariamente completa, identifica partes da propriedade que são comprovadamente satisfeitas pelo contrato e eventos no código que não podem levar a uma violação.

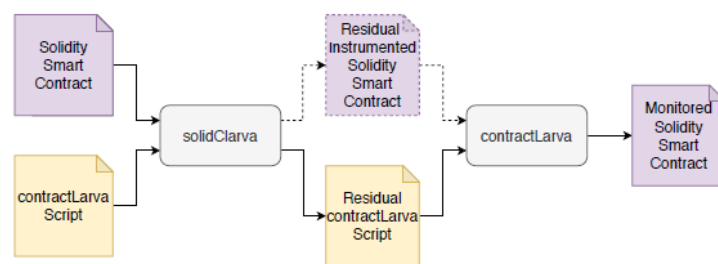


Figura 2 – Estrutura de análise do solidClarva com contractLarva

ALB: Esse é o contract larva? E chegou a olhar novamente se há algo novo? se foi disponibilizado!? Aqui tem que entrar uma comparacao/descricao do que está ruim. Se nao tem “texto” com comprovacoes do que fizeram etc. Senao nao tem como criticar e ver o que nao esta correto ou bom para ir em outra direcao.

MSN: Olhei sim, tem o repositório mas ele não é atualizado faz tempo, então continua com o mesmo problema de não gerar o contrato com a análise residual (no caso a parte do solidclarva), então a verificação não podes er completa

Como mostrado na figura, o fluxo utilizando as duas ferramentas, *solidClarva* e *contractLarva* ainda não está completo, as linhas pontilhadas mostram que a primeira ferramenta ainda não gera o contrato com a análise residual, feita pelo CFA, o que impede a verificação completa.

Outro ponto é que a abordagem feita por [2] não visa corrigir e mostrar os erros do contrato, mas sim verificar se o *smart contract* em questão segue o comportamento descrito pelo automato de propriedades, então esse método não se encaixa na presente pesquisa.

1.3 Verificação de contratos eletrônicos

Um outro trabalho abordou a alternativa para a verificação de contratos eletrônicos de forma a contornar os problemas de ambiguidade da linguagem natural e as limitações de formalismos que não identificam os participantes do contrato. A verificação de contratos por meio de linguagens formais, como a Linguagem de Contratos (CL), permite uma análise sistemática de suas cláusulas. No entanto, a CL clássica, baseada na lógica deôntica padrão, mostra-se insuficiente ao lidar com cenários multilaterais, pois não consegue determinar explicitamente quem são os responsáveis pela execução ou violação de uma determinada ação.

Para solucionar essa lacuna, [3] propôs uma extensão da CL, denominada Linguagem de Contrato Relativizada (RCL), que incorpora os conceitos da Lógica Deôntica Relativizada. A principal inovação da RCL é a capacidade de associar indivíduos a cada cláusula, especificando o portador da obrigação (quem deve agir) e a contraparte (para quem a ação é direcionada). Essa representação permite que todas as partes de um contrato multilateral sejam identificadas corretamente, permitindo uma verificação a partir das especificidades da linguagem RCL. Essa verificação é feita a partir da ferramenta RECALL.

A ferramenta RECALL opera transformando a especificação do contrato em RCL em um autômato de estados finitos. Cada estado representa uma etapa possível na execução do contrato, rotulado com as informações deônticas (obrigações, proibições) válidas naquele momento. A detecção de conflitos é então realizada através da análise do espaço de estados, onde a ferramenta busca por estados que contenham informações normativas contraditórias, como a obrigação e a proibição simultânea da mesma ação para o mesmo indivíduo.

Essa abordagem é robusta e possui uma forma simples de entender e executar. O único problema é a linguagem RCL em si. Para poder realizar a verificação, o interessado deveria ser capaz de entender e escrever um contrato de forma correta em RCL. Esse trabalho irá utilizar essa abordagem futuramente, para poder fazer a verificação dos

contratos, porém com uma metodologia diferente no quesito de escrever os contratos em RCL.a

ALB: Tem que ter uma “introducao” para o trabalho de verificacao usando RCL. Exemplo de início de texto a seguir.

ALB: “Um outro trabalho abordou a alternativa para a verificação de contratos eletrônicos ...”

ALB: Da mesma forma que introduziu os dois trabalhos anteriores. No fim, dizer qual é a direção etc.

2 FUNDAMENTAÇÃO TEÓRICA

ALB: Vai precisar uma introducao aqui mais adiante.

2.1 Contratos

Um negócio jurídico bilateral ou multilateral em que as partes envolvidas constituem, modificam ou extinguem posições jurídicas de essência ou expressão patrimonial define a noção de contrato [4]. Assim, um contrato envolve uma declaração de vontade entre as partes negociantes e atribuídos pelo ordenamento jurídico, respeitados os elementos de existência, os requisitos de validade e os fatores de eficácia contidos na norma [4].

Um contrato gera obrigações para ambas as partes envolvidas, que convencionam, por consentimento recíproco, a dar, fazer ou não fazer alguma coisa, verificando, assim, a constituição, modificação ou extinção do vínculo patrimonial [5]. Assim, um contrato deve seguir alguns princípios básicos:

Autonomia da vontade: Significa a liberdade das partes de contratar, de escolher o tipo e o objeto do contrato e de dispor o conteúdo contratual de acordo com os interesses a serem auto-regulados. Segundo [6] a liberdade de contratar pode ser vista de duas formas: pela liberdade propriamente dita, estabelecendo-se o conteúdo do contrato, ou pela escolha da modalidade do contrato, já que novas modalidades contratuais podem ser estabelecidas conforme a necessidade.

Supremacia da ordem pública: Significa que a autonomia da vontade é relativa, sujeita à lei e aos princípios da moral e da ordem pública. A supremacia da ordem pública é um dos princípios que limita a autonomia da vontade. De acordo com este princípio, o interesse da sociedade deve prevalecer sobre o interesse individual, e teve origem na constatação

de que uma liberdade irrestrita de contratar poderia ocasionar desequilíbrios e exploração do mais fraco, tornando necessária a intervenção do Estado para assegurar a igualdade econômica.

Obrigatoriedade do contrato: Significa que o contrato é garantido por lei entre as partes e os

envolvidos tem o dever da veracidade sobre o que foi acordado. Assim, um contrato não pode ser alterado

unilateralmente (*intangibilidade do contrato*), nem pode um juiz intervir nesse conteúdo, salvo em casos excepcionais [6].

2.1.1 Contratos Eletrônicos

Com a evolução da tecnologia surgiu uma forma eletrônica de lidar com contratos legais, os chamados contratos eletrônicos ou *e-contracts*. Ao contrário dos contratos tradicionais, concebidos de forma física, os contratos eletrônicos se utilizam das tecnologias computacionais para criar, negociar, armazenar e executar contratos num ambiente digital [7].

Os e-contracts são uma parte da transição de contratos tradicionais para um ambiente virtual, amplamente utilizados no cotidiano, tais como nos termos de aplicativos, cookies de websites, entre outros. Possuem diversas vantagens se comparados com contratos tradicionais, porém por estarem em uma plataforma digital, devem seguir normas e regras que adaptem esses novos contratos a leis semelhantes aos contratos tradicionais.

Os e-contracts se diferenciam dos contratos tradicionais nos seguintes aspectos [7]:

Formato digital: Esses contratos não existem na forma física, permitindo transações contínuas entre localizações geográficas distintas, além de oferecer uma maior manuseabilidade do que contratos físicos.

Comunicação Eletrônica: Os e-contracts dependem de canais de comunicação eletrônica para a troca de termos contratuais, ofertas, contrapropostas e aceitações. Isso inclui o uso de e-mails, formulários online, mensagens de chat ou outros meios eletrônicos para transmitir as intenções das partes e negociar os termos do acordo.

Assinaturas Eletrônicas: Uma característica definidora dos e-contracts é a utilização de assinaturas eletrônicas para autenticar e indicar o consentimento das partes aos termos do contrato. As assinaturas eletrônicas podem se manifestar de várias formas, como nomes digitados, assinaturas digitais, biométricas ou chaves criptográficas, dependendo do nível de segurança e requisitos regulamentares.

Validade Legal: Os e-contracts estão sujeitos a requisitos legais que regem sua validade e aplicabilidade, os quais podem variar com base na jurisdição e na natureza da transação. Esses requisitos legais frequentemente envolvem aspectos como consentimento, atribuição, integridade e não-repúdio, para garantir a autenticidade e confiabilidade dos registros eletrônicos.

Flexibilidade e Automação: Os e-contracts oferecem flexibilidade e automação na formação e execução de contratos, permitindo que as partes adaptem os termos, automatizem fluxos de trabalho e integrem processos de gestão de contratos com outros sistemas empresariais. Isso aumenta a eficiência, reduz erros e acelera o processo de contratação, resultando em um fechamento mais rápido de negócios e maior agilidade empresarial.

Para que contratos eletrônicos sejam aplicados na prática é necessário que algumas medidas prévias sejam realizadas [7]:

Autenticação: A autenticação é um componente vital para garantir a segurança de um contrato, de forma que a participação das partes envolvidas no contrato sejam validadas. Alguns métodos de autenticação são combinações de, e-mail e senha, biometria, entre outras.

Assinaturas eletrônicas: Uma vez que os contratos são digitais também deve existir uma forma de assiná-los digitalmente. Assim, a assinatura eletrônica foi criada para suprir essa necessidade, sendo realizada por meio de caixas de confirmação, criptografia ou até mesmo digitando o próprio nome.

Medidas de segurança de dados: Como contratos são documentos que podem conter informações sensíveis, é essencial garantir a segurança dos dados, com sistemas robustos de segurança envolvendo criptografia dos dados. Além disso, deve garantir o controle de leitura e escrita do contrato apenas por indivíduos autorizados.

Conformidade com requisitos legais: Os e-contracts devem respeitar um conjunto de leis especiais que contemplam o modelo. Dessa forma, os contratos devem estar sempre atualizados para que não fiquem obsoletos, tanto nos quesitos tecnológicos, quanto nos quesitos legais.

2.2 Verificação de Contratos

Erros e inconsistências de um contrato mal-formado podem trazer prejuízos a uma das partes envolvidas. Uma forma de garantir a corretude de um contrato é através de algum processo de verificação sobre as cláusulas que representam as regras estabelecidas pelo contrato. Embora os conceitos e aplicações de contratos convencionais e e-contracts apresentem diferenças, a verificação de ambos é fundamental para assegurar que nenhuma das partes envolvidas seja lesada, prevenindo possíveis conflitos e inconsistências [3].

Existem várias técnicas de análise e verificação de contratos que viabilizam um processo mais rigoroso para garantir as propriedades desejadas de um contrato. Entre essas técnicas, destaca-se o processo de negociação para contratos legais, que envolve a busca por um acordo entre as partes e que seja mutuamente aceitável. Em geral, cada parte inicia o processo propondo uma solução que atenda aos seus interesses; caso uma das partes não aceite, ocorrem contrapropostas até que ambas cheguem a um consenso. Esse processo de negociação pode se dar em contratos bilaterais ou multilaterais, podendo ou não envolver mediadores [3].

Com o contrato já negociado, uma outra etapa consiste na detecção de conflitos, uma técnica voltada para identificar e eliminar conflitos normativos em contratos.

Um conflito normativo pode invalidar o contrato e resultar em violações. Por isso, a importância de uma verificação mais precisa para se encontrar inconsistências antes que o contrato seja executado, após a fase de negociação.

Ambos os métodos podem ser eficientes na busca por conflitos, mas ainda dependem da linguagem natural na qual os contratos são escritos. Um contrato em linguagem natural pode trazer problemas de ambiguidade em suas cláusulas. Uma forma de contornar tais problemas é descrever os contratos por meio de linguagens e especificações formais, possibilitando uma verificação sistemática ao mesmo tempo que evita inconsistências [3].

A linguagem para contratos (CL) é uma dessas linguagens desenvolvidas para representar formalmente contratos legais, serviços web, interfaces e protocolos de comunicação [8]. Essa linguagem possui uma expressividade suficiente para capturar o comportamento de contratos e permitir um processo de verificação sistemático e rigoroso. A CL se baseia na lógica deôntica e na lógica dinâmica proposicional. A lógica deôntica, baseada em lógica modal [9], permite a especificação de conceitos legais e éticos, tais como obrigatoriedade, proibição e permissão, bem como a compensação para violações desses conceitos. Já a lógica dinâmica proposicional permite a especificação de sequências de ações para computar dados e representar mudanças de estado [3]. Um método de verificação de contratos utilizando a linguagem CL foi proposto por Prisacariu [8].

Outra forma de detectar problemas em contratos eletrônicos usando a linguagem CL é através da análise de conflitos normativos nas cláusulas. Esse método, proposto por Fenech [10], consiste na análise semântica de *traces*, usando autômatos e análise de alcançabilidade para identificar cláusulas conflitantes. Além disso, o método também possibilita o monitoramento dos contratos durante sua execução. A abordagem é composta das seguintes etapas:

1. tradução dos contratos para a linguagem CL;
2. decomposições das cláusulas do contrato;
3. obtenção do autômato representando as cláusulas a partir das decomposições do contrato;
4. busca por conflitos normativos no espaço de estado dos autômatos.

Embora a verificação usando tais métodos seja mais precisa do que as baseadas em linguagem natural, ainda não é possível identificar as responsabilidades sobre uma dada ação em um contrato. Em contratos bilaterais, as responsabilidades são facilmente identificadas devido a sua natureza, porém em contratos multilaterais, com dois ou mais indivíduos, a identificação das responsabilidades e violações se torna mais complexa e difícil de se lidar apenas com a linguagem CL.

Nessa linha, Herrestad et al. [11] propuseram uma extensão da lógica deôntica com relativização, de forma que os envolvidos no contrato pudessem ser identificados. A lógica deôntica relativizada personaliza os operadores deônticos, permitindo que cenários mais complexos com a participação de vários indivíduos sejam descritos de forma precisa.

Com o objetivo de buscar conflitos em contratos multilaterais e se beneficiar dos operadores deônticos relativizados, Mura [3] propôs uma extensão da linguagem CL, a RCL, adicionando à lógica deôntica relativizada na linguagem de contratos. Essa extensão adicionou a associação de indivíduos a cada ação do contrato, permitindo que as responsabilidades fossem atribuídas apenas sobre os indivíduos associados a ação. As possíveis relativizações são: (i) todos os indivíduos do contrato realizam a ação para um outro indivíduo; (ii) um indivíduo realiza uma ação para todos os outros; (iii) um indivíduo realiza uma ação apenas a um outro [12].

A sintaxe da RCL é similar à sintaxe original da CL, com os operadores deônticos acompanhados do mecanismo de penalidade em caso de violação de uma obrigação ou proibição. Na linguagem, também existem os operadores para ações concorrentes, encadeadas ou de escolha entre ações, bem como os operadores de violação ou ação dentro dos operadores deônticos. Além disso, existe o operador de iteração, indicando que uma cláusula pode ser aplicada múltiplas vezes [3].

2.3 Blockchain

O conceito de *blockchain*, base de todas as criptomoedas, foi introduzido inicialmente por Satoshi Nakamoto [13], com o objetivo de estabelecer um sistema distribuído confiável. Esse sistema, peer-to-peer, chamado Bitcoin, consiste em transações diretas entre duas partes sem a necessidade de terceiros envolvidos, dando origem ao termo *blockchain*. As blockchains, atualmente, possuem uma ampla aplicação na indústria financeira, em especial, com a introdução ao conceito de contratos inteligentes.

Nakamoto tinha o objetivo específico de criar um armazenamento distribuído de documentos, onde nenhuma das partes poderia modificar o conteúdo dos dados sem que outros participantes detectassem tal alteração [14]. A crise financeira de 2008 também colaborou para destacar a tecnologia, revelando fragilidades nos sistemas financeiros tradicionais, como a dificuldade de rastrear a propriedade de ativos em transações complexas [14].

Uma blockchain pode então ser definida como um banco de dados transacional globalmente distribuído [15]. Um usuário de uma rede blockchain pode realizar a leitura de dados da rede tais como, documentos, dados de transações, entre outros. Porém, existem transações em que todas as partes envolvidas devem estar de acordo para serem realizadas. Além disso, numa blockchain, as transações são sempre criptograficamente assinadas pelo

criador da transação [15].

As blockchains também são classificadas em três tipos diferentes, públicas, privadas e consórcio. Nas públicas, qualquer pessoa pode participar da rede e auditar as transações, porém cada transação desse contrato necessita de gas, a unidade de custo para se realizar uma operação computacional na blockchain. As redes privadas são restritas a grupos de usuários, e para ingressar na blockchain é necessário um convite, só então, é possível ter acesso ao contrato e a outros tipos de dados da rede. Já nas blockchains de consórcio, a rede é controlada por um grupo de empresas ou organizações. A diferença para redes privadas é que as redes de consórcio são parte públicas e privadas, então esse grupo de empresas controla o processo em um primeiro momento, mas outros usuários também podem participar da criação de novas transações.

2.3.1 Blocos e Transações

O termo *blockchain* tem origem na construção da rede, composta por “blocos” que são registrados em forma de cadeia. Os blocos são unidades estruturadas para armazenar dados de transações de maneira organizada e segura, sendo registrados em lotes e de maneira cronológica, evitando conflitos que possam surgir de transações simultâneas.

Cada bloco na blockchain contém:

- um timestamp, que indica o momento exato em que a transação é realizada;
- um hash do bloco anterior, que é a criptografia dos dados presentes no bloco anterior, importante para manter a cadeia do blocos;
- um nonce, que é um número aleatório usado para verificar o hash.

Quando uma transação precisa ser revertida a alteração só pode ser realizada no bloco mais recente, pois blocos subsequentes reforçam a imutabilidade de blocos anteriores. Essa característica reduz significativamente a chance de reversão ao longo do tempo devido à validação contínua da rede [14]. A organização em blocos, validada por meio de criptografia, assegura a segurança e imutabilidade da blockchain. Essa estrutura é essencial para garantir que dados armazenados sejam consistentes e que a confiança no sistema seja mantida.

Os blocos de uma blockchain são estruturadas para armazenar dados de transações de forma segura e organizada. Esses blocos garantem que as transações sejam agrupadas em lotes e registradas de forma cronológica, evitando conflitos que possam surgir de transações simultâneas. Um conflito ocorre quando duas transações tentam modificar o mesmo recurso de forma incompatível, como por exemplo, quando duas transferências tentam usar os mesmos fundos. Nesse caso, os protocolos de consenso, como o Proof of

Work (PoW) ou Proof of Stake (PoS), garantem que apenas uma transação seja validada [14, 16]. Se duas transações são conflitantes, a mais recente em termos de tempo é então invalidada, de acordo com a regra estabelecida pelo protocolo de consenso utilizado na blockchain [17, 16].

Uma blockchain possui um bloco denominado “bloco gênese”, o primeiro bloco da cadeia, esse bloco tem por padrão o numero zero e está manualmente, ao contrário dos outros blocos que vão sendo adicionados automaticamente conforme a blockchain vai crescendo, o bloco gênese é pré-definido em um arquivo de configuração ou no código do cliente da blockchain. Todos os outros blocos são gerados a partir desse bloco gênese, cada bloco contém um hash do bloco anterior, aumentando assim a blockchain [16]

Usuários de uma blockchain podem realizar diversas operações tais como, operações financeiras para transferência de criptomoedas, geração e transferência de tokens, ativos criados na blockchain que servem como meios de troca, registro de documentos, entre outras. Essas operações são realizadas por meio de transações definidas nos blocos.

Para a criação de novos blocos, um usuário deve, primeiramente, transmitir uma requisição para outro usuário da blockchain. Essa requisição é então transmitida para todos os servidores que fazem parte da blockchain (disseminação de transação). Em seguida, todos os servidores iniciam um processo de “consenso”, que consiste em avaliar os parâmetros da transação, e caso a transação seja válida, um dos servidores criará um novo bloco, adicionando a transação e transmitindo para todos os outros servidores. Estes servidores devem então validar e adicionar o novo bloco a sua blockchain [16].

Essa estrutura de transações mantém a integridade e segurança da blockchain, garantindo que todas as transações sejam válidas e que a blockchain seja imutável [14].

Note que a rede utiliza um mecanismo de consenso para validar as transações e adicionar novos blocos à cadeia, assegurando que alterações nos dados sejam detectáveis e que o sistema permaneça seguro e confiável.

2.3.2 Smart Contracts

O surgimento da blockchain deu origem aos *smart contracts*, conceito introduzido por [18], que combina protocolos computacionais com interfaces de usuário para executar automaticamente os termos de um contrato. Com a tecnologia blockchain, esses contratos passaram a ser implementados de forma mais acessível e segura, substituindo intermediários tradicionais como advogados e instituições financeiras [14].

Os *Smart contracts* podem gerenciar tanto a propriedade de ativos tangíveis quanto intangíveis, automatizando transações complexas e minimizando a necessidade de confiança em terceiros, o que pode transformar radicalmente setores inteiros da economia.

Para que um *smart contract* seja escrito, é necessário que seus objetivos, requisitos,

partes envolvidas, descrição dos estados e eventos que irão ocorrer no contrato, estejam bem definidos. Em seguida, um programador escreve tais informações usando uma linguagem de programação específica, como por exemplo a Solidity, uma linguagem de alto nível voltada para a implementação de *smart contracts*.

A linguagem permite que todas as condições e ações que o contrato deve realizar sejam codificadas de forma clara e segura, garantindo que o contrato seja executado de acordo com o esperado [15]. Após a codificação do contrato, este pode ser então implantado na blockchain, sendo identificado por um endereço único para ser acessado por qualquer parte interessada do contrato.

2.4 Smart Contracts

Anteriormente, a forma mais digitalizada de um contrato era o contrato eletrônico, que mantinha praticamente todas as características de um contrato em papel, mas em um ambiente virtual. No entanto, como destacado por [18], os contratos eletrônicos ainda dependiam de intermediários para validação e execução. Em contraste, os smart contracts são um conjunto de promessas, especificadas em formato digital, incluindo protocolos pelos quais as partes cumprem essas promessas [18]. Essa abordagem elimina a necessidade de intermediários e garante a execução automática e segura dos termos contratuais.

O termo *Smart Contract* foi concebido, inicialmente, por Nick Szabo, na década de 90. O objetivo era descrever contratos em um código que pudesse ser alheio às partes do contrato e fosse autoaplicável com suas cláusulas [18]. Essa visão surgiu da ideia de que a lógica contratual poderia ser transformada em software, mantendo a ideia do contrato em ambiente virtual, porém removendo a necessidade de intermediários para garantir a execução dos termos acordados.

Um *smart contract* é um agente inteligente e/ou um programa de computador capaz de tomar decisões quando determinadas precondições são atendidas [19]. A inteligência de um agente depende da complexidade da transação programada para ser realizada. Os contratos podem ser transações muito simples, executadas em segundos ou minutos, ou transações relativamente complexas e demoradas, que envolvem negociações e dezenas de páginas de texto escrito com direitos e obrigações específicas, podendo levar horas ou meses para serem concluídas. Atualmente, os *smart contracts* se enquadram na categoria de transações relativamente simples.

2.4.1 Aspectos Legais

Um smart contract é considerado legalmente válido quando atende aos requisitos básicos do direito contratual tradicional. Isso inclui a intenção de vinculação das partes, oferta e aceitação claras, capacidade legal dos envolvidos e cumprimento das leis aplicáveis.

Sem esses elementos, um contrato inteligente pode ser contestado e considerado inválido em um tribunal. A falta de linguagem jurídica clara e a dificuldade de interpretar o código como prova em disputas judiciais são desafios que ainda precisam ser enfrentados [19].

Além disso, as diferenças regulatórias entre países podem afetar a aplicabilidade dos *smart contracts* em contextos internacionais [19]. Por exemplo, enquanto alguns países aceitam contratos digitais como equivalentes aos contratos escritos, outros ainda precisam de uma estrutura legal que reconheça explicitamente contratos baseados em código. Essas diferenças exigem um esforço contínuo para alinhar os desenvolvimentos tecnológicos com a legislação vigente, garantindo que os *smart contracts* possam ser aplicados de forma justa e eficaz.

Embora os *smart contracts* ofereçam a promessa de contratos autoexecutáveis, vários desafios legais significativos ainda persistem. A imutabilidade dos *smart contracts* é um problema recorrente, já que qualquer alteração nas cláusulas do contrato exige que um novo contrato seja criado e verificado, gerando custos adicionais [19]. Ainda assim, os *smart contracts* ainda se mostram mais vantajosos que os contratos tradicionais, pois no caso de contratos tradicionais, uma verificação manual das partes envolvidas é necessária, para que as mudanças sejam analisadas e aceitas, demandando ainda mais retrabalho do que em contratos inteligentes.

2.4.2 Execução dos Smart Contracts

A execução de um *smart contract* ocorre de maneira automática, conforme as condições predefinidas. Como o contrato é codificado, essas condições devem ser estabelecidas, bem como possíveis restrições. Após a criação e codificação, o contrato deve ser implantado na *blockchain*, para que as operações implementadas sejam executadas de acordo com as cláusulas definidas.

Uma vez que o contrato se encontra disponível na *blockchain*, para que qualquer uma das partes possa utilizar as funções do contrato, uma transação na *blockchain* deve ser aberta, contendo sua autoria, a mesma que consta no contrato. Quando as condições codificadas no contrato são atendidas, automaticamente as ações previstas devem ser executadas. Por exemplo, a execução de uma ação pode transferir ativos, liberar informações ou registrar dados. Essa automação é uma das principais vantagens dos *smart contracts*, pois elimina a necessidade de intermediários, reduz custos e agiliza processos, aumentando a eficiência e transparência sobre as transações realizadas [20].

No entanto, a execução automatizada de *smart contracts* também apresenta desafios, especialmente quando ocorrem erros no código ou quando surgem situações não previstas durante a fase de desenvolvimento. Vale lembrar que uma característica importante dos *smart contracts* é que, uma vez implantados na *blockchain*, alterações ou correções não podem ser facilmente realizadas. Essa imutabilidade garante a confiança na

execução do contrato, mas também implica em riscos. Assim, a imutabilidade dos *smart contracts* tem suas vantagens, como por exemplo assegurar a execução correta dos termos, mas também suas desvantagens, como por exemplo, perpetuar falhas não detectadas antes de sua implantação. Porém, ao mesmo tempo, se o contrato é imutável, pode perpetuar suas falhas, se um *smart contract* contém um defeito de interpretação das cláusulas propostas, ao ser implementado na blockchain, essa falha pode continuar até que esse contrato não seja mais útil, então sua imutabilidade pode ser perigosa se esse tipo de situação não for detectada.

Para evitar que a imutabilidade dos contratos não seja um problema, é necessário que exista um mecanismo confiável para se obter uma interpretação precisa dos requisitos de um contrato para sua implementação. Um dos mecanismos possíveis é a detecção de conflitos em contratos, usando linguagens formais e lógicas matemáticas para representar um contrato de forma precisa, evitando possíveis conflitos e inconsistências no contrato [3].

Apesar dos desafios, os *smart contracts* continuam a evoluir como uma ferramenta poderosa para a execução automatizada de acordos, com potencial para transformar diversas indústrias. A flexibilidade limitada, as complexidades de verificação e as incertezas legais são obstáculos que necessitam de atenção contínua. No entanto, à medida que as práticas de desenvolvimento de *smart contracts* avançam e a legislação se adapta, espera-se que esses contratos se tornem cada vez mais integrados ao sistema legal e comercial global.

2.5 Blockchain de Propósito Geral

ALB: Precisa uma introducao para as 3 subsecoes

2.5.1 Ethereum

O surgimento do Bitcoin abriu caminho para um novo segmento da tecnologia. Em 2015 foi então lançada uma nova plataforma universal de aplicações baseadas na blockchain, o Ethereum. Essa plataforma trouxe uma nova proposta de aplicações, provendo praticamente todo tipo de computação através dos *smart contracts* [21]. O Ethereum possui as características de uma blockchain, definida por seus blocks e transações, com o foco principal nos *smart-contracts*.

No Ethereum existem dois tipos de contas que podem realizar essas transações: contas externas e contas de *smart contracts*. Essas contas são associadas com programas em uma linguagem de alto nível usando uma máquina virtual chamada de Ethereum Virtual Machine (EVM). As contas externas são controladas por pares de chaves públicas e privadas, podendo enviar transações para outras contas. Já as contas de *smart contracts* são controladas pelo código armazenado junto a essas contas. Esse código é então

executado quando a transação associada é acionada na conta [22].

Uma conta possui um conjunto determinado de informações: um *Nonce*, que indica o número de transações enviadas pela conta, ou a quantidade de criações de outros *smart contracts* feitas (em caso de contratos de contratos), e garante que cada transação criada pela conta seja processada corretamente; *Balance*, determina a quantidade de *wei* (a menor quantidade de ethereum: $1 \text{ ether} = 10^{18} \text{ wei}$) controlado por essa conta; e *CodeHash*, que possui um código em bytecode do contrato presente nas contas; quando a conta é externa esse valor é vazio [21].

Essas contas são geradas com endereços de 160 bits de duas maneiras: para contas externas o endereço é derivado da chave pública; já no caso de contas de contrato o endereço é gerado a partir do endereço do proprietário e do nonce associado.

O Ethereum também possui dois tipos de transações: do próprio contrato quando introduzidos na rede blockchain, e as transações por chamada de mensagem, que executam alguma função de um contrato já existente, ou para transferir ether. Uma característica importante de transações na rede Ethereum é que cada ação necessita de um *gas*, que é uma medida em ether. Para cada transação, o usuário que a enviou deve informar o *gas limit*, a quantidade máxima de *gas* que aquela ação deve consumir, e o *gas price*, o preço que o usuário gostaria de pagar por unidade de *gas* [22]. Quando a transação é efetivada, o preço final é obtido pelo *gas limit* multiplicado pelo *gas price*. Se alguma quantidade de *gas* não é utilizada, o valor pode então ser reembolsado. Porém, se não houver saldo ou *gas* na conta para realizar a transação, ou se ocorrer um erro no contrato da conta, todo o *gas* enviado é consumido [21].

2.5.2 Solidity

Solidity é uma linguagem de programação de alto nível desenvolvida para descrever contratos inteligentes para a plataforma Ethereum [23]. Inicialmente, os contratos para o Ethereum eram escritos na linguagem LLL (Low-level Lisp-like Language), Serpent (linguagem baseada em python), e Viper (outra linguagem baseada em python). Porém, como a maior parte dos desenvolvedores de *smart contracts* eram desenvolvedores web [23], a solução foi desenvolver uma linguagem baseada em javascript para atrair mais usuários, facilitando a curva de aprendizado da linguagem, tornando Solidity uma das linguagens mais utilizadas no Ethereum.

A linguagem Solidity é baseada no paradigma orientado a objetos, onde a representação de um contrato se assemelha a declaração de uma classe [23]. Além disso, a linguagem permite princípios como o de herança e polimorfismo. A herança permite que um contrato herde propriedades e métodos de outro contrato, usando a palavra-chave *is* para estabelecer a relação de herança [15]. O polimorfismo permite que um contrato base chame uma função de um contrato derivado, mesmo que a função tenha sido sobrescrita (

quando uma função em um contrato derivado redefine uma função existente no contrato base, mantendo o mesmo nome e parâmetros). Esse princípio é útil quando um contrato derivado precisa se comportar de maneira diferente do contrato base.

A Solidity é uma linguagem estaticamente tipada, ou seja, os tipos das variáveis são verificados em tempo de compilação, antes que o programa seja executado. Isso significa que o tipo de cada variável (por exemplo, `int`, `string`, `bool`) deve ser declarado explicitamente ou inferido pelo compilador, e não pode ser alterado durante a execução do programa [15].

Outro recurso importante da linguagem são os modificadores (*modifiers*), que permitem alterar o comportamento de funções de forma declarativa [15]. Esses modificadores são comumente utilizados para verificar automaticamente condições específicas antes da execução de uma função. Quando um mesmo trecho de verificação se repete em múltiplas funções de um contrato, é possível criar um modificador para centralizar essa lógica e aplicá-lo a todas as funções que necessitam da mesma validação. O modificador não apenas reforça a segurança do contrato, mas também elimina duplicação de código. Além dessa aplicação clássica, os modificadores podem ser usados para executar trechos de código antes ou depois da função principal (utilizando o marcador “`_`”) ou pode chamar outras funções internas do contrato.

Como um *smart contract* é baseado na relação entre duas ou mais partes, é importante entender como o código em Solidity se relaciona com os acordos feitos entre essas partes. Em geral, um contrato gera obrigações entre as partes envolvidas, que convencionam, por consentimento recíproco, a dar, fazer ou não fazer alguma coisa. Os participantes de um contrato, por exemplo, são variáveis do tipo *address* declaradas, que representam endereços no Ethereum. Já as obrigações são implementadas como funções que exigem ações específicas, tal como realizar pagamentos, que pode ser descrito por meio de modificadores ou por outros métodos. Cláusulas de um contrato que possuem prazos, termos de pagamento e responsabilidades podem ser expressas por meio de combinações de recursos da linguagem, como funções com modificadores, funções que mudam o valor de variáveis, entre outras. As punições por descumprimento de alguma cláusula podem ser implementadas por meio de restrições, baseadas no endereço da conta do punido, ou reverter funções que detectaram um descumprimento ou até mesmo banimento do usuário punido daquele contrato. Através desses recursos é possível então descrever praticamente qualquer acordo entre duas ou mais partes de um contrato em Solidity.

A Solidity também possui o conceito de evento que permite um contrato se comunicar com eventuais usuários e aplicações, além de realizar a função de registro das transações realizadas no contrato. Quando um evento é acionado, um registro da transação é associado ao endereço do contrato, facilitando o acesso de fora da blockchain [24]. O contrato a seguir exemplifica os recursos mencionados.

```

1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.0;
3
4 // Contrato base (pai)
5 contract Animal {
6     string public name;
7
8     // Evento para registrar quando um animal emite um som
9     event SoundEmitted(address indexed animalAddress, string sound);
10
11     constructor(string memory _name) {
12         name = _name;
13     }
14
15     function speak() public virtual returns (string memory) {
16         string memory sound = "Animal sound";
17         emit SoundEmitted(address(this), sound); // Emite o evento
18         return sound;
19     }
20 }
21
22 // Contrato derivado (filho) que herda de Animal
23 contract Dog is Animal {
24     constructor(string memory _name) Animal(_name) {}
25
26     function speak() public override returns (string memory) {
27         string memory sound = "Woof!";
28         emit SoundEmitted(address(this), sound); // Emite o evento
29         return sound;
30     }
31 }
32
33 // Outro contrato derivado (filho) que herda de Animal
34 contract Cat is Animal {
35     constructor(string memory _name) Animal(_name) {}
36
37     function speak() public override returns (string memory) {
38         string memory sound = "Meow!";
39         emit SoundEmitted(address(this), sound); // Emite o evento
40         return sound;
41     }
42 }

```

Listagem 1 – Contrato simples em Solidity

O programa em Solidity apresentado é composto por três contratos: Animal, Dog e Cat. Os dois últimos herdam características do contrato Animal, no qual o evento “SoundEmitted” e a função `speak`, que contém o modificador *virtual* no contrato Animal, indica que essa função pode ser sobrescrita. Já os contratos Dog e Cat chamam a função `speak`, usando o modificador *override* e sobrescrevendo-a para retornar “Woof!” ou “Meow!”. Em seguida, emitem o evento `soundEmitted` que indica o endereço de onde o evento foi chamado e o som que deve emitir. Todas as funções indicam o tipo do seu retorno.

2.5.3 Remix IDE

Um *smart contract* pode ser executado quando é implantado numa blockchain. No caso, um contrato em Solidity pode ser implantado na plataforma do Ethereum para ser executado. Uma ferramenta que simula o Ethereum é a Remix IDE, que auxilia no desenvolvimento dos *smart contracts* escritos em Solidity.

A Remix IDE fornece um ambiente virtual onde desenvolvedores podem escrever, depurar e testar seus contratos [25]. A ferramenta inclui funcionalidades como realce de sintaxe, detecção de erros básicos e suporte à compilação. Além disso, na plataforma simulada, as transações não requerem aprovação externa, o que facilita a execução de testes. Para simular transações, a Remix VM disponibiliza contas pré-configuradas, cada uma com 100 ETH, permitindo que os desenvolvedores testem suas implementações pagando o custo de gas associado a cada operação.

Para executar um contrato na Remix VM é necessário, primeiramente, compilá-lo para garantir que não haja erros sintáticos no código. Após a compilação bem-sucedida, a opção de implantação na blockchain é habilitada. Durante a implantação, o desenvolvedor pode especificar a quantidade de ETH ou wei que o contrato deve receber. Uma vez implantado, o desenvolvedor pode interagir com as funções do contrato, realizando transações e testando seu comportamento. O terminal integrado da IDE exibe detalhes de cada transação, incluindo se ela foi aceita ou rejeitada, se algum erro ocorreu ou se eventos foram emitidos [25].

Outra funcionalidade importante da IDE é o suporte a testes unitários, que permite aos desenvolvedores criar e executar testes para garantir a integridade do contrato. Esses testes são baseados no comportamento esperado das funções do contrato, ajudando a identificar possíveis falhas ou inconsistências. Após a criação do arquivo de teste, sua execução verifica se todas as condições estabelecidas foram atendidas ou se algum teste falhou. Em caso de falha, a IDE fornece detalhes permitindo que o desenvolvedor analise e corrija o problema. A Remix IDE também oferece ferramentas de depuração, que permite inspecionar o comportamento do contrato durante a execução dos testes, facilitando a identificação e resolução dos erros [25].

2.6 Redes de Petri

O modelo de Redes de Petri foi proposto por Carl Adam Petri [26] com o objetivo de especificar sistemas paralelos, concorrentes, assíncronos e não-determinísticos. Esse modelo formal permite uma análise precisa, como a verificação de propriedades e da correteza do sistema [27].

2.6.1 Redes de Petri Ordinárias

As redes de Petri, originalmente propostas por Petri [26] são formadas por um conjunto de transições e lugares. Os lugares correspondem às variáveis de estado e as transições às ações do sistema. Uma ação pode ser realizada se as condições associadas a alguma variável de estado forem satisfeitas. Logo, uma transição tem uma relação direta com um ou mais lugares. Após a realização de uma transição, alguns lugares tem seus valores alterados como consequência. Os lugares e transições são representadas graficamente por círculos e retângulos, respectivamente, como ilustrado na Figura 3 [27].

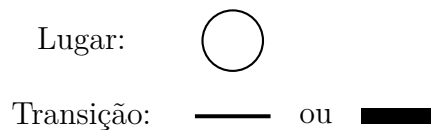


Figura 3 – Elementos de um Rede de Petri.

Os lugares e as transições são associados por meio de arcos direcionados. Um lugar ligado à uma transição significa uma condição necessária para que a transição seja disparada. Já uma transição ligada à um lugar significa que após a execução dessa transição, o lugar é alterado. Além disso, esses arcos também podem ser ponderados, indicando a quantidade de marcas consumidas ou geradas nos lugares. As marcas (ou *tokens*) são informações distribuídas nos lugares, determinando a marcação (ou estado) da rede em um determinado momento. A semântica de uma Rede de Petri é dada pela mudança da marcação associada aos lugares ao longo da execução da rede.

Uma rede complexa pode então ser modelada por subredes, ou seja, redes mais básicas que, combinadas, descrevem o fluxo de uma aplicação complexa. Essas redes básicas representam a modelagem de (a) um sequenciamento, quando um lugar habilita uma transição e então avança para um novo lugar; (b) uma distribuição, quando um lugar habilita uma transição que dispara em paralelo, avançando para dois novos lugares; (c) uma junção quando dois lugares habilitam uma única transição, avançando para um outro lugar; e (d) um não-determinismo, quando um lugar habilita duas transições, mas o disparo de uma delas impede o disparo da outra [28].

ALB: Aqui deverá entrar as figuras dessas estruturas básicas, se for o caso.

MSN: ok

A partir desse conceito de redes e subredes, surgem padrões estruturais importantes, como o *fork* e o *join*, que gerenciam a concorrência. Uma estrutura *fork* (bifurcação) ocorre quando uma única transição possui um lugar de entrada e múltiplos lugares de saída. Esse padrão representa paralelismo: quando a transição dispara, o marcador de seu lugar de entrada é consumido e um marcador em cada um de seus lugares de saída é produzido, permitindo assim múltiplas atividades paralelas. Em contrapartida, com a

estrutura de um *join* (junção) múltiplos caminhos paralelos são sincronizados. A estrutura consiste em uma transição com múltiplos lugares de entrada e apenas um lugar de saída. A transição só é habilitada quando todos os lugares de entrada possuem os marcadores necessários. Ao disparar, a transição consome os marcadores de cada um desses lugares e produz um único marcador em seu lugar de saída, realizando a sincronização da rede.

Um exemplo de Rede de Petri ordinária é dado na Figura 4, que modela o funcionamento de uma máquina de lavar roupas. Os lugares “ProntaParaLavar”, “Lavando” e “RoupaLavada” representam os diferentes estados do sistema durante o processo de lavagem. Já os lugares “RoupaSuja”, “ÁguaDisponível” e “SabãoDisponível” representam as condições necessárias para que o processo seja iniciado, considerando um marcador em cada lugar.

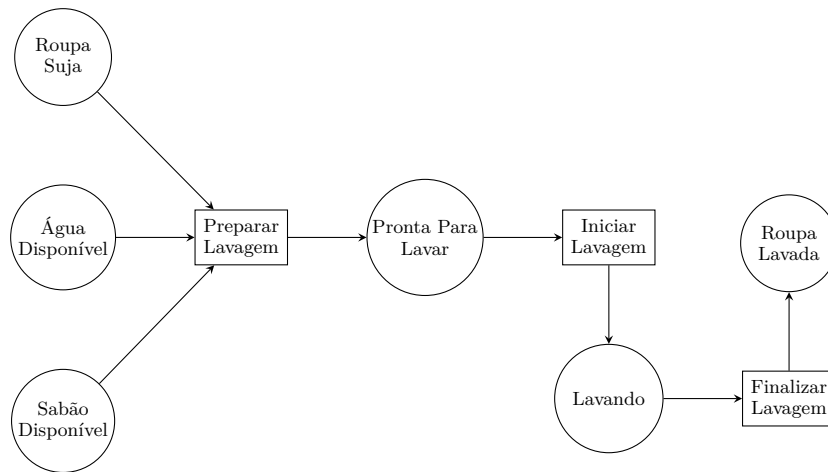


Figura 4 – Rede de Petri de uma máquina de lavar.

A transição “PrepararLavagem” então fica habilitada quando todos os lugares de condição (pré-condições) estiverem marcados com marcadores, ou seja, quando roupa, água e sabão estiverem disponíveis. Quando essa transição é disparada, os marcadores das condições são consumidos, e um novo marcador é gerado no lugar “ProntaParaLavar”, representando o avanço do sistema para o próximo estado. O processo prossegue habilitando a transição “IniciarLavagem”, que quando disparada gera um marcador no lugar “Lavando”, indicando que o processo de lavagem está ocorrendo. O marcador desse lugar habilita a transição “FinalizarLavagem”, que ao disparar consome o marcador do lugar “Lavando” e gera um marcador no lugar “Roupa Lavada”, terminando o processo.

A descrição formal de uma Rede de Petri ordinária é dada pela Definição 1.

Definição 1 Uma Rede de Petri R é dada pela tupla (P, T, A, V, K) , onde:

- $P = \{p_1, p_2, p_3, \dots, p_m\}$ é o conjunto finito de lugares;
- $T = \{t_1, t_2, \dots, t_n\}$ é o conjunto finito de transições;

- $A \subseteq (P \times T) \cap (T \times P)$ é o conjunto de arcos que interligam lugares às transições ou transições aos lugares;
- V é a função de valoração dos arcos dada por $V : A \rightarrow \mathbb{N}$, onde cada arco (p, t) (de entrada) ou (t, p) (de saída) tem um inteiro positivo atribuído. Caso não haja um valor associado é assumido o peso 1 ($V(a) = 1$). Assim, um arco de entrada tem o valor definido por $V_e(p, t)$, que representa o peso associado ao arco do lugar de entrada p para a transição t . De forma similar, a função $V_s(t, p)$ define o valor associado da transição t para o lugar de saída p ;
- $K : P \rightarrow \mathbb{N}$ é o conjunto de marcações de uma rede. De forma geral, define-se uma marcação K como uma função que mapeia o conjunto de lugares P em inteiros não-negativos, representando a distribuição de marcas (estado) da rede em um determinado momento. Assim, $K(p)$ indica a quantidade de marcas no lugar $p \in P$ e K_0 representa a marcação inicial da rede ($K_0 : P \rightarrow \mathbb{N}$, no momento inicial). Logo, $K_0(p)$ indica a quantidade inicial de marcas no lugar $p \in P$.

Já a semântica de uma Rede de Petri é definida pelas regras que governam sua evolução dinâmica. Esse comportamento é determinado pelo fluxo de marcadores através da rede, que ocorre por meio dos disparos das transições. Para que uma transição seja habilitada em uma rede ordinária, é necessário que todos os seus lugares de entrada contenham ao menos o número de marcadores igual ao peso do arco associado. Uma vez que essa condição é satisfeita, a transição pode disparar, consumindo os marcadores de cada lugar de entrada conforme definido pelo peso do arco, produzindo um ou mais marcadores em cada um de seus lugares de saída, também de arco com o peso do arco que conecta a transição a esses arcos. Esse ciclo de habilitação e disparo constitui o fluxo contínuo da rede.

A semântica de uma Rede de Petri é dada pela Definição 2 e pode então ser dividida em dois aspectos: a condição de habilitação, que determina se uma transição pode disparar com base na marcação atual da rede; e a regra de disparo, que define como a marcação da rede é alterada ao consumir e produzir marcadores, movendo o sistema para um novo estado.

Definição 2 *Seja $R = (P, T, A, V, K)$ uma Rede de Petri ordinária, a semântica de R é definida por:*

1. **habilitação:** uma transição $t_j \in T$ está habilitada para uma dada marcação K se, e somente se, todos os seus lugares de entrada contêm a quantidade de marcadores exigida pelo peso de seus respectivos arcos. Formalmente, a habilitação de t_j por uma marcação K é denotada por $K \models t_j$, e a condição é satisfeita se:

$$\forall p_i \in P, K(p_i) \geq V_e(p_i, t_j)$$

onde $K(p_i)$ é a quantidade de marcadores no lugar p_i e $V_e(p_i, t_j)$ é o peso associado ao arco do lugar p_i para a transição t_j , ou seja, o peso para o lugar de entrada. A marcação habilita o disparo, contudo não o obriga a disparar.

2. **disparo:** quando uma transição habilitada, t_j , dispara, os marcadores de seus lugares de entrada são consumidos e novos marcadores são produzidos em seus lugares de saída, movendo o sistema para uma nova marcação K' . Denota-se a mudança da marcação de K para K' pelo disparo de t_j por $K \xrightarrow{t_j} K'$. A nova marcação K' é obtida pela equação de estado:

$$\forall p_i \in P, K'(p_i) = K(p_i) - V_e(p_i, t_j) + V_s(p_i, t_j),$$

onde $V_s(p_i, t_j)$ representa o peso do arco da transição t_j para o lugar p_i , ou seja, o peso para o lugar de saída.

Considere a Rede de Petri ordinária da máquina de lavar roupas da Figura 4. Assuma peso 1 para todos os arcos e uma marcação inicial $K_0 = (1, 1, 1, 0, 0, 0)$ indicando que os insumos estão prontos.

A transição *PrepararLavagem* tem *RoupaSuja*, *AguaDisponível* e *SabaoDisponível* como lugares de entrada. A condição de habilitação da transição é dada por $K_0(p) \geq V_e(p, \text{PrepararLavagem})$ para cada $p \in \{\text{RoupaSuja}, \text{AguaDisponível}, \text{SabaoDisponível}\}$. Como a marcação atual de cada um desses lugares é 1 e o peso do arco de entrada (V_e) exigido para cada um também é 1, a condição é satisfeita ($1 \geq 1$). Portanto, a transição *PrepararLavagem* encontra-se habilitada para a marcação inicial. As transições *IniciarLavagem* e *FinalizarLavagem*, de outra forma, não estão habilitadas, pois seus lugares de entrada (*ProntaParaLavar* e *Lavando*) possuem 0 marcadores na marcação inicial.

Quando uma transição habilitada, como é o caso de *PrepararLavagem*, é disparada, a marcação da rede muda de K_0 para uma nova marcação K_1 . O disparo consome os marcadores de *RoupaSuja*, *AguaDisponível* e *SabaoDisponível*, produzindo um novo marcador no lugar de saída (*ProntaParaLavar*), já que o peso do arco de saída $V_s(\text{PrepararLavagem}, \text{ProntaParaLavar})$ é 1. Aplicando a equação de estado, o novo vetor de marcação é dado por $K_1 = (0, 0, 0, 1, 0, 0)$. Esse novo estado (K_1) cria uma nova pré-condição para habilitar o disparo da próxima transição, *IniciarLavagem*.

2.6.2 Redes de Petri coloridas

Alguns tipos de sistemas podem apresentar problemas mais específicos, exigindo um maior detalhamento, ou lidando com condições complexas. Nestes cenários, extensões das redes de Petri ordinárias são abordadas, como por exemplo, as redes de Petri coloridas.

Nas redes ordinárias, existe apenas um tipo de marca, logo é preciso várias marcas e diferentes lugares para modelar um sistema mais complexo, deixando a rede grande e complexa para ser gerenciada. Outra limitação nas redes clássicas é a impossibilidade de hierarquização, ou seja, não é possível abstrair comportamentos para sub-redes, o que torna a representação desses sistemas utilizando redes ordinárias muito mais trabalhoso, e resulta em uma rede final complexa se comparado à mesma representação utilizando uma rede colorida [27].

As redes de Petri coloridas facilitam a especificação de sistemas mais complexos através do conceito de cores. As marcas são individualizadas através de cores associadas, onde cada cor é um tipo diferente e único de dado [29]. Dessa forma, diferentes processos ou recursos podem ser modelados em uma mesma sub-rede. Cada lugar pode armazenar diversas marcas e cada uma dessas marcas tem um valor associado a um conjunto de dados mais complexo. Além disso, cada lugar tem um conjunto de cores (Σ) associados, que funciona como um tipo de dado composto [27]. Logo, todas as marcas de um lugar específico devem seguir a estrutura pré-definida, embora cada marca possa ter um valor (cor) único. Por exemplo, no lugar **Pedidos**, o conjunto de cores define uma estrutura com os atributos **tipo** e **tempo**; assim, uma marca individual (cor) pode ser (**tipo**="expresso", **tempo**=10), enquanto outra pode ser (**tipo**="prioritário", **tempo**=20).

Nas redes coloridas cada lugar, transição ou arco possui inscrições associadas, anotações textuais que fornecem informações adicionais e controlam o comportamento do modelo. Os lugares possuem três tipos de inscrições: nome, conjunto de cores e expressão de inicialização (marcação inicial). As transições possuem dois tipos de inscrições: nome e expressões guardas. Já os arcos possuem apenas um tipo de inscrição, dado por uma expressão de guarda.

A dinâmica das redes de Petri coloridas também muda, pois agora o disparo de transições depende das marcações e suas cores associadas. As expressões de guarda possuem condições booleanas para o disparo, como por exemplo [**prioridade** == "alta"], e as expressões nos arcos definem como as marcas são consumidas/produzidas, como por exemplo, um arco entre um lugar **Fila** e uma transição **Atender** com a expressão $1'(\text{tipo}=\text{"expresso"})$ indica que apenas marcas dessa cor, neste caso "expresso", podem ser consumidas.

Uma Rede de Petri Colorida é formalmente dada pela Definição 3.

Definição 3 *Uma Rede de Petri Colorida (RPC), R_c , é dada por $(P, T, A, \Sigma, C, G, V, In)$, onde:*

- *P é o conjunto finito de lugares da rede. $P = \{p_1, p_2, p_3, \dots, p_n\}$.*
- *T é o conjunto finito de transições. $T = \{t_1, t_2, \dots, t_n\}$.*

- $A \subseteq (P \times T) \cap (T \times P)$ é o conjunto de arcos que interligam lugares às transições ou transições aos lugares;
- Σ é o conjunto finito, não-vazio de tipos, denominados cores;
- $C : P \rightarrow \Sigma$ é a função de coloração que associa uma cor a cada lugar, tal que $C(p) \subseteq \Sigma$.
- G é a função de guarda dada por $G : T \rightarrow \text{exp}$ que associa uma expressão exp a uma transição $t \in T$. Assim, $G(t)$ é uma expressão de guarda e $\text{Tipo}(\text{Var}(G(t))) \in \Sigma$ retornam todos os tipos das variáveis usadas em $G(t)$.

Note que $\text{Var}(\text{exp})$ retorna o conjunto das variáveis em exp e $\text{Tipo}(v)$ retorna o conjunto de cores de todas as variáveis $v \in \text{Var}(G(t))$.

- $V : A \rightarrow \text{expr}$ é a função de expressão de arco que associa a cada arco $a \in A$ uma expressão booleana. De forma similar às redes ordinárias, define-se $V_e(p, t)$ como a expressão do arco de entrada (do lugar p para a transição t) e $V_s(t, p)$ como a expressão do arco de saída (da transição t para o lugar p). A expressão é satisfeita se resultar em um multiconjunto de cores compatível com o conjunto de cores definido para o lugar p adjacente ao arco, ou seja, $\forall a \in A, \text{Tipo}(V_e(a)), \text{Tipo}(V_s(a)) = C(p)$. Além disso, todas as variáveis contidas na expressão devem possuir tipos pertencentes ao conjunto global de cores $\text{Tipo}(\text{Var}(V_e(a))), \text{Tipo}(\text{Var}(V_s(a))) \subseteq \Sigma$. Essa definição assegura que as marcas consumidas ou produzidas por um arco pertençam obrigatoriamente ao universo de dados permitido para um lugar específico.
- $\text{In} : P \rightarrow \text{expr}$ é a função de inicialização que define a marcação inicial dos lugares por meio de expressões fechadas (sem variáveis). Para garantir a consistência do modelo, o tipo da expressão deve resultar em um multiconjunto de cores compatível com o conjunto de cores definido para um lugar específico ($\forall p \in P, \text{Tipo}(\text{In}(p)) = C(p)$). O requisito de que o conjunto de variáveis da expressão seja vazio ($\text{Var}(\text{In}(p)) = \emptyset$) assegura que o estado inicial da rede seja composto obrigatoriamente por valores constantes ou expressões que resultem em valores fixos e conhecidos antes de qualquer disparo de transição.

A Figura 5 ilustra uma rede de Petri colorida para um restaurante. O conjunto de tipos de dados (cores) é dado por $\Sigma = \{\text{CLIENTE}, \text{PRATO}, \text{COMANDA}\}$. CLIENTE é um tipo de dado que representa os clientes (ex: C1, C2), PRATO é um tipo que representa os itens do cardápio (ex: Pizza) e COMANDA é um tipo composto pelo produto (CLIENTE $\times$ PRATO) que associa um cliente a um prato (ex: (C1, Pizza)).

A rede de Petri colorida para o restaurante é então definida por:

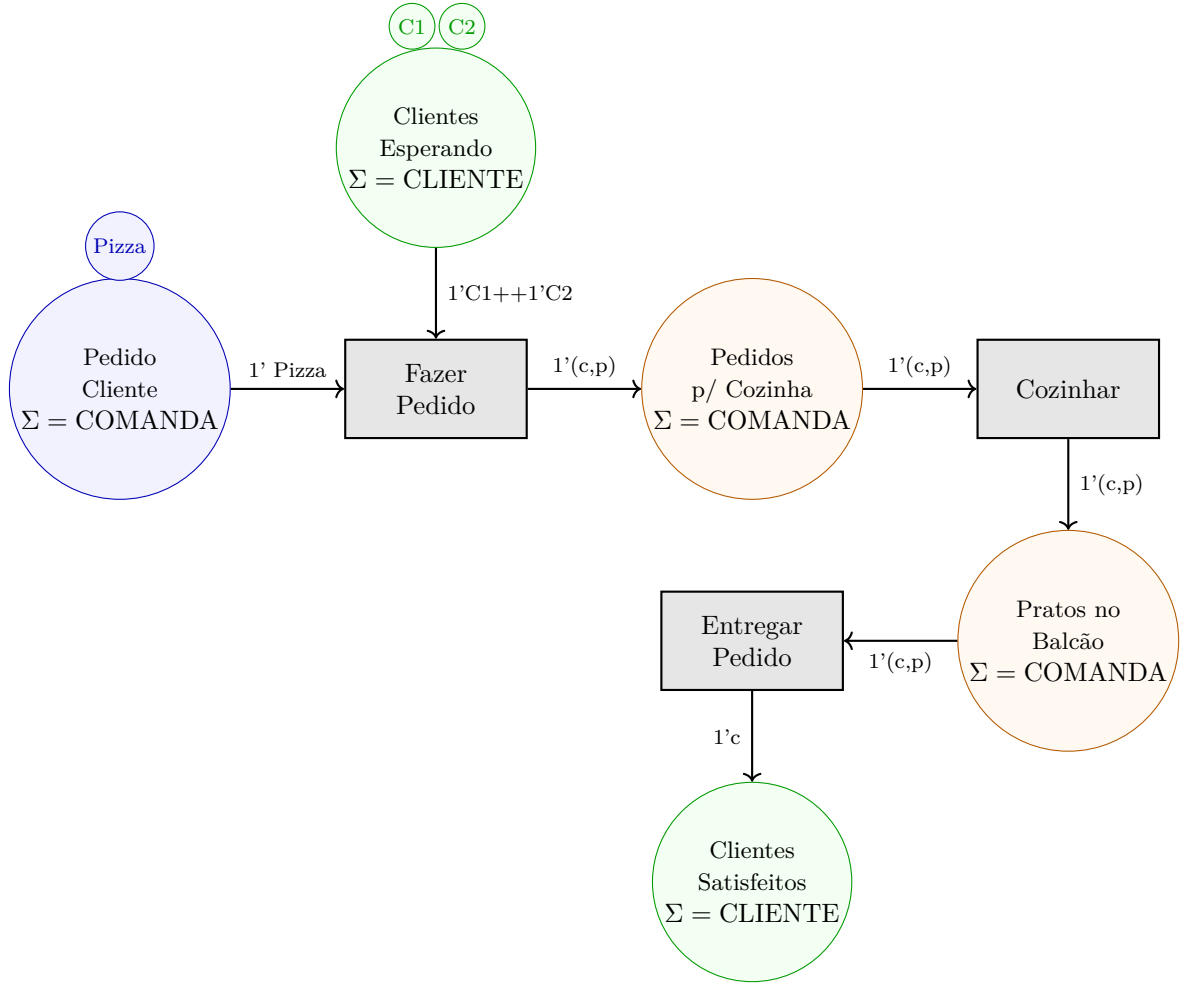


Figura 5 – Rede de Petri colorida para um restaurante.

- $P = \{\text{Clientes Esperando}, \text{Pedido Cliente}, \text{Pedidos p/ Cozinha}, \text{Pratos no Balcão}, \text{Clientes Satisfeitos}\}.$
- $T = \{\text{Fazer Pedido}, \text{Cozinhar}, \text{Entregar Pedido}\}.$
- $\Sigma = \{\text{CLIENTE}, \text{PRATO}, \text{COMANDA}\}$, onde COMANDA é o produto de $\text{CLIENTE} \times \text{PRATO}$.
- A função C associa um tipo a cada lugar, como por exemplo $C(\text{Clientes Esperando}) = \text{CLIENTE}$, $C(\text{Pedido Cliente}) = \text{PRATO}$, e $C(\text{Pedidos p/ Cozinha}) = \text{COMANDA}$.
- A função In define a marcação inicial da rede com $In(\text{Clientes Esperando}) = 1'C1 ++ 1'C2$, onde dois clientes, $C1$ e $C2$, estão esperando, e $In(\text{Pedido Cliente}) = 1' \text{ Pizza}$, enquanto os demais lugares iniciam vazios. Na notação de multiconjuntos em RPCs, o símbolo $'$ separa o coeficiente (quantidade) do valor da cor, e o operador $++$ representa a adição de multiconjuntos, ou seja, a união de marcas de diferentes cores no mesmo lugar. Assim, o exemplo apresentado indica que há um marcador do cliente $C1$ e um marcador do cliente $C2$ esperando para ser atendido.

- As funções V_e e V_s definem as expressões de arcos sobre as variáveis da rede. Assim, para $c \in CLIENTE$ e $p \in PRATO$, o arco da transição de **Fazer Pedido** para o lugar **Pedidos p/ Cozinha** possui a expressão de saída associada definida por $V_s(\text{FazerPedido}, \text{Pedidos p/ Cozinha}) = (c, p)$, que estabelece a produção de um novo marcador do tipo *COMANDA*.
- A função de guarda G é trivial (implicitamente **true**) para todas as transições, pois não há condições lógicas adicionais que restrinjam o disparo além da disponibilidade de marcadores.

Já a semântica da rede de Petri é obtida por sua execução a partir da marcação inicial, através do pedido de um cliente. Note que a transição *FazerPedido* só pode disparar quando os lugares de entrada *ClientesEsperando* e *PedidoCliente* possuem marcadores disponíveis. Além disso, as funções V_e e V_s estabelecem as exigências para o disparo nos respectivos arcos. O arco de entrada direcional do lugar *ClientesEsperando* para a transição *FazerPedido* possui a expressão $V_e(\text{ClientesEsperando}, \text{FazerPedido}) = 1'C1 + 1'C2$, enquanto o arco a partir de *PedidoCliente* tem a expressão $V_e(\text{PedidoCliente}, \text{FazerPedido}) = 1'Pizza$.

Quando a transição *FazerPedido* dispara, um marcador de cada lugar de entrada é consumido, associando valores concretos às variáveis por meio do processo de amarração. No exemplo da Figura 5 a variável c assume o valor *C1* e a variável p assume o valor *Pizza*. Com esses valores vinculados, a expressão do arco de saída (c, p) é avaliada, resultando na produção de um novo marcador composto, $1'(C1, Pizza)$ do tipo *COMANDA*, que é então depositado no lugar de saída *PedidosParaCozinha*.

A semântica da RPC é estabelecida formalmente na Definição 4.

Definição 4 *Seja uma Rede de Petri Colorida $R_c = (R, \Sigma, C, G, V, In)$, a semântica de R_c é dada por:*

- **Amarração (Binding):** *Atribui valores específicos (cores) às variáveis de uma transição. Para uma transição t com variáveis $v_1, \dots, v_n$, uma amarração b válida associa a cada variável v_k um valor c_k pertencente ao seu tipo (cor). Assim, uma transição t está habilitada para uma amarração b se:*
 1. *a expressão de guarda $G(t)$ é verdadeira para a amarração b .*
 2. *cada lugar de entrada p da transição t possui as marcas requeridas pela expressão do arco de entrada $V_e(p, t)$, quando avaliada com a amarração b .*

*Na Figura 5, a transição **FazerPedido** possui as variáveis c e p . Uma amarração válida associa c ao valor *C1* e p ao valor *Pizza*.*

- **Habilitação:** Uma transição $t \in T$ está habilitada para uma amarração específica b se ambas as condições a seguir forem satisfeitas:
 1. A expressão de guarda $G(t)$ é avaliada como *true* para a amarração b .
 2. Cada lugar de entrada p da transição t possui os marcadores requeridos pela expressão do arco de entrada $V_e(p, t)$, quando avaliada com a amarração b .
- **Disparo:** Uma transição t pode disparar quando habilitada para a amarração b . O disparo altera a marcação de M para M' da seguinte forma:
 1. **Consumo:** para cada lugar de entrada p_i de t , são consumidos os marcadores especificados pela avaliação da expressão de arco $V_e(p_i, t)$ com a amarração b .
 2. **Produção:** para cada lugar de saída p_k de t , são produzidos os marcadores especificados pela avaliação da expressão de arco $V_s(t, p_k)$ com a amarração b .

Essa abordagem permite que as marcas produzidas tenham valores que são o resultado de uma função sobre os valores consumidos.

A semântica das redes de Petri coloridas, assim como nas ordinárias, é definida pela regra de disparo das transições. Contudo, as regras para uma rede colorida é significativamente mais rica, pois o disparo não depende apenas da existência de marcas, mas também dos seus valores e tipos (cores), além das condições lógicas adicionais, conhecidas como guardas. Com base nessas regras semânticas, podemos entender como os padrões e extensões das redes ordinárias são expressos de forma mais poderosa em RPCs.

As estruturas de *fork* e *join*, por exemplo, mantêm sua função de gerenciar o paralelismo e a sincronização. Contudo, em uma RPC, o fluxo de informação é muito mais rico. Um *fork* pode não apenas iniciar processos paralelos, mas também distribuir informações específicas e relacionadas a cada um desses processos, a partir de uma mesma amarração. Já o *join* pode realizar uma sincronização muito mais inteligente, exigindo não apenas a presença de marcadores, mas que esses marcadores estejam relacionados pelo mesmo valor (cor), algo que é verificado na avaliação das expressões de arco $V(p, t)$ durante a habilitação.

Da mesma forma, o conceito de arcos ponderados é generalizado em RPCs através de multiconjuntos (ou bags). Um multiconjunto é uma coleção de elementos onde cada valor (cor) pode aparecer múltiplas vezes, permitindo representar quantidades variadas de itens distintos em uma única expressão. Enquanto uma rede ponderada ordinária usa apenas um número inteiro para especificar a quantidade de marcadores idênticos, a expressão de arco $E(p, t)$ em uma RPC utiliza a notação *valor'tipo* para indicar a quantidade (*valor*) de uma cor específica (*tipo*), unindo-as pelo operador $++$. A expressão $1'Pizza++2'Pasta$, por exemplo, exige ou produz, simultaneamente, uma marca do tipo “Pizza”

e duas do tipo “Pasta”. Essa flexibilidade permite modelar o consumo ou a produção de itens heterogêneos, indo muito além da simples contagem de marcadores anônimos.

Assim, a semântica das RPCs, fundamentada na amarração de variáveis, eleva os conceitos das redes mais simples:

1. o paralelismo do *fork* passa a ser um fluxo de informações coordenadas;
2. a sincronização do *join* torna-se uma junção baseada em dados;
3. o peso dos arcos evolui de uma mera contagem para uma manipulação explícita de conjuntos de dados complexos.

3 DE CONTRATOS INTELIGENTES PARA REDES DE PETRI

Este capítulo propõe a transformação de contratos escritos em Solidity para modelos de Redes de Petri pode ser realizada em três etapas [30]. As duas primeiras etapas estão relacionadas a leitura e obtenção dos dados de contrato, enquanto a terceira, de fato, constrói a rede de Petri com base nesses dados de contrato.

As próximas seções descrevem as etapas de transformação de um contrato escrito em Solidity para uma Rede de Petri Colorida.

ALB: *A ver outras secoes*

3.1 Uma Abordagem de Transformação

A primeira etapa consiste em obter as variáveis globais do contrato e todas as informações associadas de forma organizada e estruturada. Informações importantes como nome, tipo, valor, tipo índice (para mappings) e nível de visibilidade (public ou private), são gravadas em uma tabela com todas as variáveis globais. Além disso, as funções do contrato também são armazenadas constando seus respectivos nomes, variáveis de retorno, tipos das variáveis de retorno, nível de visibilidade da função, nome e tipo das variáveis de parâmetro e a linha de início e fim da função. Nesta fase os dados extraídos são organizados na tabela *Variáveis Globais*, que armazena o nome (N), o tipo (T), o valor inicial (V), a visibilidade (VI) e o tipo do índice (TI) para variáveis indexadas como mapeamentos ou vetores. A tabela *Funções* consolida os metadados das rotinas identificadas, registrando o nome da função, as listas de nomes e tipos de retorno, a visibilidade, e as variáveis de parâmetro com seus respectivos tipos, além de delimitar o escopo usando a numeração das linhas de código de início (I) e fim (F).

Na sequência, as operações contidas no escopo de cada função, tais como atribuições, cálculos matemáticos e expressões lógicas, são analisadas e armazenadas através de registros de dados. Esses registros descrevem a variável que recebe o resultado (V), a operação propriamente dita (O) e o conjunto de variáveis que representam os operandos (P). Além da lógica interna, as chamadas de função também são incluídas em uma tabela específica, que armazena o nome da função invocada, os parâmetros passados e a linha de código onde ocorreu a chamada.

Complementarmente, a tabela *Chamadas* rastreia as invocações de subfunções, armazenando o nome da função chamada (N), as variáveis passadas como argumentos (P) e a linha exata (L) da instrução no código original, permitindo o mapeamento posterior de dependências na rede formal. Já as estruturas de controle são organizadas na tabela

Condicionais, que armazena o tipo de comando (T), tais como *if*, *require* e *assert*, a expressão lógica da condição (C) e o intervalo de linhas (I e F) que define o bloco de código afetado pela regra de negócio.

A segunda etapa consiste na transformação dos elementos armazenados nas tabela para elementos de uma Rede de Petri. Esse mapeamento induz, primeiramente, a criação dos lugares da rede, usando a tabela de variáveis globais. Assim, cada variável global se torna um lugar da rede, e os tipos de cada variável atribui uma certa cor para cada lugar, e os valores das variáveis determinam as marcações iniciais.

As funções do código Solidity também são organizadas em tabelas e dão origem as transições da Rede de Petri. Algumas funções também dão origem a lugares que representam parâmetros de uma função interna, usando o prefixo *par* - seguido pelo campo N do registro. Funções que interagem com o ambiente externo ao contrato, como as funções de leitura de dados de oráculos (serviços que fornecem dados externos à *Blockchain*), geram um tipo especial de lugar, o lugar oráculo. Este lugar representa a entrada de dados externos à rede, permitindo que a semântica da RPC capture a dependência do contrato sobre informações que não estão diretamente na *Blockchain*, modelando a interação com o mundo real.

Já as operações dão origem aos arcos de entrada e saída das transições, enquanto as chamadas de função são modeladas como arcos especiais entre a transição que representa a chamada e a transição correspondente à função chamada. Esses arcos também conectam os lugares às transições e vice-versa, e cada um possui uma expressão de arco que define o tipo e a quantidade de marcas que são consumidos ou produzidos. Os arcos de entrada a uma transição definem as marcas, tanto as cores quanto sua quantidade, devem estar disponíveis no lugar de origem para que a transição seja habilitada. Já os arcos de saída determinam a geração das marcas após o disparo da transição nos lugares de destino.

A linguagem Standard ML (SML) [31] permite que essas expressões de arco sejam bastante elaboradas, refletindo a lógica complexa de manipulação de dados que ocorre nas funções do contrato Solidity. Essa linguagem é utilizada em ferramentas como o CPN Tools para definir os tipos de cores, variáveis, expressões de arcos e guardas [32]. Por exemplo, uma operação de atribuição ($x = y + z$) em Solidity pode ser traduzida para arcos de entrada dos lugares y e z para a transição que representa a operação, e um arco de saída dessa transição para o lugar x , com uma expressão de arco que soma os valores das marcas de y e z , resultando na quantidade de marcas somadas no lugar x .

Além das variáveis, funções e operações, os contratos inteligentes em Solidity frequentemente utilizam modificadores de funções (*modifiers*) para alterar o comportamento e verificar condições restritivas previamente à execução de um bloco de código. Na transformação para RPC, a lógica desses modificadores é mapeada diretamente para expressões de guarda (função G) associadas às transições ou para expressões de arco (função E).

Quando traduzidos para expressões de guarda, os modificadores atuam como restrições booleanas rigorosas que devem ser avaliadas como verdadeiras para que a transição correspondente à função seja habilitada e possa disparar. Por outro lado, quando mapeados para expressões de arco, os modificadores definem restrições sobre os valores específicos das marcas que a transição está autorizada a consumir ou produzir. No exemplo do restaurante, um modificador que restringisse a execução da ação “fazer pedido” apenas para o cliente *C1*, seria encapsulada como uma restrição de acesso através da expressão de guarda [*cliente* == *C1*]. Assim, a transição “FazerPedido” só estaria habilitada se a marca consumida satisfizesse as exigências booleanas impostas pelo modificador no contrato original.

3.2 Arquitetura da Transformação

A abordagem de transformação apresentada na Subsecção 3.1 é sistematizada através do paradigma de programação orientada a objetos, em duas etapas bem específicas. Na primeira etapa é realizada a análise sintática e semântica do contrato Solidity, enquanto que a segunda etapa consiste no mapeamento do modelo de objetos para as componentes da Rede de Petri Colorida.

A etapa de análise sintática e semântica do contrato Solidity é realizada com o suporte da ferramenta ANTLR¹, que recebe um arquivo *.g4* com a definição da gramática da linguagem em questão. Neste caso, o arquivo define a sintaxe da linguagem de programação Solidity, com suas palavras-chave, como por exemplo *contract* e *function*, os tipos de dados, tais como *address* e *uint*, os operadores existentes e a estrutura dessas componentes combinadas. Uma vez que esta gramática é fornecida ao ANTLR, o código-fonte de um analisador léxico e um analisador sintático é automaticamente gerado para alguma linguagem de programação como Java ou C#.

A árvore sintática (AST²) gerada pela ferramenta representa o código-fonte estruturado conforme a abordagem descrita na Subsecção 3.1. O objetivo é projetar um algoritmo que explore esta árvore usando o padrão de projeto *Visitor*. As informações relevantes são extraídas de cada nó da árvore e armazenadas em classes do modelo de objetos que representam a lista de variáveis globais e as funções do contrato. Por exemplo, um nó da AST pode ser uma *functionDefinition* ou uma *variableDeclaration* com informações importantes como o nome, o tipo e sua visibilidade.

O modelo é composto por classes que especificam as variáveis globais, com seus atributos representando o tipo de dado, valor e visibilidade, as funções com seus parâmetros e código, além das operações internas. As dependências entre partes do código são especificadas pelas relações entre as classes do modelo. A classe “Função” deve ser

¹ *ANother Tool for Language Recognition*

² *Abstract Syntax Tree*

associada à classe “Operação”, indicando que uma função possa conter várias operações, e essas operações, por sua vez, deve manter referências às variáveis. O resultado desta etapa gera um modelo de objetos que armazena de forma organizada toda a informação estrutural do contrato escrito em Solidity.

A segunda etapa consiste no mapeamento do modelo de classes para componentes de uma Rede de Petri Colorida. O objetivo é também representar essas componentes da rede através de classes de objetos. Os Lugares (*Places*) devem ser especificados por classes que definem seus atributos como o tipo de cor (*Color Set*) e sua marcação inicial. As cores podem ser eficientemente mapeadas por meio de tipos enumerados (*enums*). Já as transições (*Transitions*) são geradas em associação com as operações e funções do contrato. A lógica interna de cada operação determina a estrutura e as expressões dos arcos de entrada e saída da transição correspondente. Em cenários de maior complexidade, as próprias expressões dos arcos podem ser encapsuladas em classes específicas para um melhor gerenciamento de sua construção.

Outro tipo de comando que apresenta uma certa dificuldade na transformação é o comando `revert`. Em Solidity, o comando `revert` é uma instrução de manipulação de erro que interrompe uma execução e desfaz todas as mudanças de estado realizadas durante a chamada do contrato e suas subchamadas [15]. Uma mudança de estado em Solidity refere-se à alteração de valores em variáveis de estado, que são dados armazenados de forma permanente na memória do contrato dentro da blockchain [15]. Na RDP, existe uma dificuldade intrínseca em modelar esse comportamento já que o formalismo padrão não armazena o histórico de marcações anteriores para realizar restaurações automáticas e não possui um mecanismo para reconhecer quais operações devem ser anuladas. Embora o comportamento possa ser representado, a rede de Petri precisaria de uma arquitetura complexa de sub-redes dedicadas para armazenar o estado atual e transições específicas para restaurar esses marcadores em caso de falha.

ALB: Aqui então já estamos adiantando que não vamos lidar com o `revert`, por exemplo? MSN: Por enquanto sim, se conseguirmos lidar sem problemas aí posso retirar esse parágrafo e colocar como fizemos ou coisa assim

Para sumarizar a operacionalização da transformação do modelo de objetos de Solidity para as componentes e semântica das Redes de Petri Coloridas, o processo analítico pode ser delineado nas seguintes etapas:

Análise Sintática e Geração do Modelo (ANTLR): O código-fonte do contrato é lido para gerar um modelo estrutural (Árvore de Sintaxe Abstrata) de onde se extrai as informações de variáveis, funções e estruturas customizadas. No exemplo do restaurante, esta etapa deve identificar a declaração do enumerador `Cliente` e seus possíveis valores, `C1` e `C2`.

Mapeamento de Lugares (Places) e Cores (Color Sets): As variáveis globais e tipos enumerados definem os lugares da rede e seus respectivos domínios de valores, ou seja, as cores. No cenário do restaurante, a estrutura de dados enum Cliente cria formalmente um *Color Set* $\Sigma = \{C1, C2\}$, instanciando lugares onde as marcas armazenadas assumirão obrigatoriamente um desses dois valores.

Mapeamento de Transições: Os blocos de execução e as funções extraídas do contrato Solidity são convertidos em transições da RPC. Caso o exemplo do restaurante possua métodos para “reservar mesa”, a função se torna uma transição aguardando as marcas.

Definição de Expressões de Arco e Expressões de Guarda: O fluxo de negócio, as operações (matemáticas ou lógicas) e os modificadores são traduzidos em regras de consumo e geração nas expressões de arco, ou como delimitadores booleanos nas expressões de guarda das transições. Se a função de reserva do restaurante altera o estado do cliente *C1*, a rede geraria um arco do lugar “Cliente” para a transição da reserva consumindo especificamente a marca do tipo *C1*.

3.3 Algoritmo de Transformação

O Algoritmo 1 descreve o módulo principal da transformação de um contrato Solidity em uma RPC usando as estruturas de dados definidas e as duas principais funções de transformação: o analisador de código do contrato em Solidity; e a construção da RPC usando as classes de objetos. O algoritmo proposto se utiliza dos conceitos de orientação objetos, como o diagrama de classes, para facilitar a organização e o processo de transformação.

Algoritmo 1: De Solidity para Rede de Petri Colorida (RPC)

```

1 ListasInfo: VariaveisGlobais, Funcoes, Operacoes, Chamadas, Condicionais.
2 RPC: Classe de objeto da Rede de Petri com Lugares, Transicoes, Arcos.
3 Function TransformarContrato(códigoFonteSolidity)
4     // Fase 1: Análise do Contrato
5     ListasInfo  $\leftarrow$  AnalisarCodigo(códigoFonteSolidity);
6     // Fase 2: Construção da RPC
7     RPC  $\leftarrow$  ConstruirRPC(ListasInfo);
8     return RPC;
9 end
```

O módulo principal da transformação então consiste na extração das informações em duas classes principais, *ListasInfo* e *RPC*, que relacionam o código Solidity com o modelo formal da rede de Petri. A estrutura *ListasInfo* é o repositório de metadados da análise estática com suas propriedades dada por listas de objetos especializados. As principais coleções dessa classe incluem:

- *VariaveisGlobais*: armazena objetos com propriedades tais como nome (N), tipo (T), valor inicial (V), visibilidade (VI) e o tipo do índice (TI), para variáveis complexas como mappings ou vetores.
- *Funcoes*: contém registros com o nome da função, lista de parâmetros (Parametro-Funcao), tipos de retorno e o escopo delimitado pelas propriedades de linha inicial (I) e final (F).
- *Operacoes*: Captura especificamente as atribuições e cálculos, armazenando a variável de destino (V), o operador (O), operandos utilizados (P) e a linha do código referente ao contrato onde ocorreu a operação (L).
- *Chamadas*: Rastreia as invocações de subfunções, armazenando de forma estruturada o nome da função chamada (N), o conjunto de variáveis passadas como argumentos (P) e a linha correspondente da chamada (L) no código original do contrato.
- *Condicionais*: Armazena os comandos de controle de fluxo e validação (como *if*, *require* e *assert*). Cada registro encapsula o tipo de comando (T), a expressão lógica da condição (C), a linha inicial (I) e a linha final (F) que delimitam o escopo da restrição.

A Rede de Petri Colorida obtida é estruturado por meio do paradigma orientado a objetos através de uma classe principal que agrega e gerencia três coleções de objetos: Lugares, Transições e Arcos. Cada componente da rede é mapeado como uma classe específica que encapsula suas propriedades estruturais:

Lugar: Representa uma variável de estado da rede usando o atributo *Id* para identificar unicamente um lugar. Possui o atributo nome (*Name*) derivado da variável existente no contrato, e a sua marcação inicial (*InitialMarking*), que define o estado da rede antes de qualquer disparo.

Transição: Representa as funções e ações do contrato através de um identificador único (*Id*), o nome da ação (*Name*) e uma expressão de guarda (*Guard*), que atua como condição lógica baseada nos modificadores e regras de negócio que ditam a habilitação do disparo.

Arco: Representa a conexão entre lugares e transições, estabelecendo o fluxo e a topologia da rede. Possui um identificador (*Id*), uma referência direta ao objeto origem (*SourceId*) e ao objeto destino (*TargetId*), além de abrigar a expressão matemática (*Expression*) que dita as regras de consumo ou geração de marcas.

O Algoritmo 2 descreve a função *AnalisarCodigo* que usa a biblioteca ANTLR4 para eliminar ambiguidades do contrato Solidity e gerar um modelo estruturado através de uma árvore sintática de informações.

Algoritmo 2: Análise do Código Solidity usando ANTLR

```

1 Function AnalisarCodigo(códigoFonte)
2   ListasInfo ← InicializarListas();
3   AST ← GerarAST(códigoFonte) ;                               // Usa-se um parser como ANTLR
4   foreach nó n na AST do
5     if n é uma Declaração de Variável Global then
6       regVar ← Extrair dados da variável de n;
7       Adicionar regVar em ListasInfo.VariaveisGlobais;
8     if n é uma Definição de Função then
9       regFunc ← Extrair dados da função de n;
10      Adicionar regFunc em ListasInfo.Funcoes;
11      foreach instrução s no corpo da função n do
12        if s é uma Operação then
13          Adicionar registro em ListasInfo.Operacoes;
14        else if s é uma Chamada de Função then
15          Adicionar registro em ListasInfo.Chamadas;
16        else if s é um Condicional then
17          Adicionar registro em ListasInfo.Condicionais;
18        end
19      end
20    else
21      end
22  end
23  return ListasInfo;
24 end

```

O algoritmo está dividido nas seguintes etapas:

1. Análise Léxica (Lexer): O componente SolidityLexer realiza a varredura do arquivo .sol, transformando o fluxo de caracteres em uma sequência de símbolos, tais como as palavras-chave *contract*, *function*, identificadores e operadores.
2. Análise Sintática (Parser): O SolidityParser recebe esses símbolos e, com base nas regras gramaticais definidas no arquivo *Solidity.g4*, constrói a Árvore de Sintaxe Abstrata (AST). Esta árvore fornece uma representação hierárquica completa de todas as estruturas do contrato.
3. Visitação da Árvore (*SolidityVisitor*): A extração dos dados dessa árvore ocorre através da classe *SolidityVisitor.cs*, que implementa o padrão de projeto *Visitor* para realizar a busca por nós da *AST*. O Algoritmo 2 descreve essa busca (linhas 8 a 22). Na implementação do algoritmo, as verificações genéricas, sobre o tipo de nó, descritas no pseudocódigo são materializadas através da sobreposição (*override*) dos métodos nativos gerados pela biblioteca do ANTLR.

O *SolidityVisitor* utiliza métodos específicos para capturar cada contexto. Por exemplo, a condição descrita na linha 5 do algoritmo (“se *n* é uma Declaração de Variável Global”) é executada pelo método *VisitStateVariableDeclaration* que extrai os dados

e preenche a lista correspondente em *ListasInfo*. ALB: A explicação não está batendo com o algoritmo. Acho que está acontecendo isso pois parte é de implementação, e está misturando com algoritmo. MSN: Não entendi, qual parte não está batendo? Seria o caso de explicar como extrai os dados e guarda em *ListasInfo*?

ALB: A linha 11 do algoritmo não tem nada sobre “*VisitFunctionDefinition*”. Ou é sobre outro algoritmo? MSN: corrigido De forma análoga, na linha 8 o algoritmo implementa o método *VisitFunctionDefinition*, que processa o escopo das funções e construtores.

Durante essa busca, o *visitor* mantém um estado interno (*funcaoAtual*) para garantir que as operações e chamadas de função sejam associadas corretamente ao seu contexto de execução. Por fim, o mapeamento das instruções internas (linhas 14 a 17 do algoritmo) é realizado por métodos como *VisitExpressionStatement* e *VisitIfStatement*, que identificam e classificam atribuições, invocações de outras funções e estruturas de controle de fluxo, instanciando os registros de operações e condicionais que servirão de base para as expressões de arco da RPC.

ALB: Acho que vai ter que levar isso para a implementação. Deixar só a parte do algoritmo mesmo.

ALB: Ok, mas onde está isso? Por exemplo, voce coloca no texto *VisitFunctionDefinition*, *VisitStateVariableDeclaration*, e nao tem nada disso no algoritmo. Mais a frente tambem retoma estes termos, e nao conseguimos ver isso no algoritmo. Se é algo interno (do algoritmo/biblioteca pronta da AST), acho que teria que dizer apenas o que se obtem dali, mas sem referenciar termos “internos” da biblioteca.

MSN: Expliquei melhor, é que as funções são geradas automaticamente por classes internas da biblioteca, e não temos acesso direto a elas, o que da pra fazer é o override delas pra especificar algum comportamento que você quer. ALB: Certo. Então duas alternativas. Ou consideramos que é interno (da biblioteca e só apontamos para algum lugar que explique ela), ou então tem que levar para a parte de implementação. Deixar como está vai sempre deixar ruído no texto.

MSN: ok, acho que talvez seria melhor deixar como interno e explicar, pois essa parte é uma abstração que retira bastante da complexidade da implementação

ALB: Ok, pode ser. Melhor refatorar isso então e/ou jogar isso para implementação como dito antes.

O *SolidityVisitor* utiliza métodos específicos para cada tipo de nó encontrado na árvore. A função *VisitStateVariableDeclaration* extrai os dados das variáveis de estado e preenche a lista correspondente na *ListasInfo* e a função *VisitFunctionDefinition* processa o escopo das funções e construtores. Um estado interno (*funcaoAtual*) também é mantido para garantir que operações e chamadas de função sejam associadas corretamente ao seu contexto de execução. Já a função *VisitExpressionStatement* e *VisitIfStatement* identificam e classificam atribuições, invocações de outras funções e estruturas de controle de

fluxo, instanciando os registros de operações e condicionais que servirão de base para as expressões de arco da RPC.

Por fim, o Algoritmo 3 transforma as variáveis e parâmetros em *Lugares*. Para cada entrada na tabela *VariaveisGlobais*, um lugar é instanciado com o nome da variável como identificador. O tipo de dado determina o conjunto de cores (multiconjunto) e o valor atribuído define a marcação inicial. O algoritmo também gera lugares para representar os parâmetros das funções. Caso uma função seja pública ou externa, o lugar de parâmetro é rotulado como um oráculo, indicando a possibilidade de receber dados externos à *blockchain*.

O algoritmo percorre a lista de *Funcoes* para gerar as *Transições* da rede, onde cada transição recebe o nome da função correspondente do código Solidity. A geração de arcos e suas respectivas expressões é a etapa mais complexa, onde o algoritmo estabelece a dinâmica da rede:

- *Fluxo de Variáveis*: para variáveis globais, o algoritmo cria arcos duplos (um de entrada e um de saída da transição), onde o arco de entrada representa o uso do dado e o de saída reflete a atualização do estado da variável após a execução.
- *Chamadas de Função*: as invocações de subfunções são transformadas em arcos que conectam a transição chamadora ao lugar que representa os parâmetros da função chamada, simulando a passagem de argumentos.
- *Lógica de Negócio*: os registros de *Operacoes* e *Condicionais* são transformados em expressões de arco. Essas expressões definem o consumo e produção de marcas, a linguagem Standard ML para facilitar a compatibilidade com ferramentas como o CPN Tools.

A última etapa do Algoritmo 3 é encarregada de transformar a lógica de negócio extraída do código Solidity em expressões matemáticas. O algoritmo processa os arcos de entrada, realizando o mapeamento das variáveis de arco conforme o conjunto de cores (multiconjuntos) do lugar de origem. Cada variável de arco deve ser única em uma mesma transição para evitar ambiguidades no consumo de marcas. Para os arcos de saída, os registros das tabelas de *Operações* e *Condicionais* são usados para construir as expressões de arcos que refletem as mudanças de estado: atribuições como $x = y + z$ são traduzidas em funções que combinam os valores dos marcadores consumidos para gerar o novo valor no lugar de destino. Por fim, comandos de controle de fluxo e validação, como *if*, *require* e *assert*, são transformados em condicionais lógicas nos arcos (ou expressões de guarda), que condicionam a produção de novas marcas de acordo com as regras de negócio, simulando o comportamento de bloqueio ou reversão de transações presente no ambiente do Ethereum.

Algoritmo 3: Construção da Rede de Petri Colorida (RPC)

```

1 Procedure ConstruirRPC(ListasInfo) is
2   RPC ← InicializarRPC();
   // Passo 1: Criar Lugares
3 foreach var em ListasInfo.VariaveisGlobais do
4   | lugar ← Criar Lugar a partir de var;
5   | Adicionar lugar em RPC.Lugares;
6 end
7 foreach func em ListasInfo.Funcoes do
8   | if func possui parâmetros then
9   | | lugarParam ← Criar Lugar para os parâmetros de func;
10  | | Adicionar lugarParam em RPC.Lugares;
11  | end
12 end
   // Passo 2: Criar Transições
13 foreach func em ListasInfo.Funcoes do
14  | transicao ← Criar Transição com o nome de func;
15  | Adicionar transicao em RPC.Transicoes;
16 end
   // Passo 3: Criar Arcos
17 foreach func em ListasInfo.Funcoes do
18  | if func possui parâmetros then
19  | | criar Arco (LugarParam -> TransicaoFunc); foreach var em
20  | | | ListasInfo.VariaveisGlobais do
21  | | | | if var é utilizada em func (verificado em Operações/Condicionais) then
22  | | | | | Criar Arco Duplo (LugarVar <-> TransicaoFunc);
23  | | | | end
24  | | end
25 end
26 foreach call em ListasInfo.Chamadas do
27  | if var é utilizada em func (verificado em Operações/Condicionais) then
28  | | transChamadora <- IdentificarTransicaoPorLinha(call.Linha);
29  | | lugarDestino <- BuscarLugarParametros(call.Nome);
30  | | Criar Arco (transChamadora -> lugarDestino);
31  | end
32 end
   // Passo 4: Gerar Expressões de Arco e Guardas
33 foreach arco em RPC.Arcos do
34  | if arco.Direcao == Entrada then
35  | | arco.Expressao <- MapearVariaveisPeloConjuntoDeCores(arco.Origem);
36  | end
37  | else
38  | | op <- BuscarOperacaoRelacionada(arco.Destino, call.Linha);
39  | | cond <- BuscarCondicionalRelacionada(arco.Destino, call.Linha);
40  | | arco.Expressao <- GerarExpressaoSML(op, cond);
41  | end
42 end
43 return RPC;
44 end

```

A função *Criar Arco* é implementada para estabelecer conexões unidirecionais entre lugares e transições. No contexto dos parâmetros de funções, a função liga o lugar que representa os parâmetros (identificado pelo prefixo par-) à transição que representa a própria função. Já a função *Criar Arco Duplo* é essencial para lidar com variáveis globais. Quando uma função usa uma variável de estado, o algoritmo cria dois arcos: um de entrada (do lugar para a transição), simbolizando a leitura do dado, e outro de saída (da transição para o lugar), representando a restituição ou atualização do valor da variável após a execução da função.

A função *MapearVariaveisPeloConjuntoDeCores* atua no mapeamento dos arcos de entrada e consiste em identificar o conjunto de cores (multiconjunto) do lugar de origem do arco e associar variáveis únicas de arco para cada tipo de dado presente nesse conjunto. O objetivo é garantir que cada variável de arco seja única dentro de uma mesma transição, evitando ambiguidades no consumo de *marcas* e assegurando que a união desses tipos resulte exatamente no multiconjunto permitido para aquele lugar.

As funções *BuscarOperacaoRelacionada* e *BuscarCondicionalRelacionada* realizam consultas às tabelas de metadados geradas pelo Algoritmo 2, na primeira etapa da transformação. A função *BuscarOperacaoRelacionada* localiza na tabela “Operações” os registros de atribuições e cálculos matemáticos (operandos e operadores) vinculados a uma variável e linha de código específicas. De forma análoga, a função *BuscarCondicionalRelacionada* consulta a tabela “condicionais” para identificar estruturas de controle que regem aquele trecho do código. Ambas fornecem a lógica de construção das expressões dinâmicas da rede.

A função *GerarExpressaoSML* é a etapa final da lógica de negócio, traduzindo as operações e condições recuperadas para a linguagem SML, utilizada em ferramentas de Redes de Petri como o CPN Tools. Sua implementação converte atribuições do Solidity, como por exemplo $x = y + z$, em funções matemáticas que processam os valores das marcas consumidas para gerar novos valores nos lugares de saída. Além disso, essa função incorpora as condicionais como expressões de guarda ou lógica interna do arco, garantindo que o fluxo das marcas na RPC reflita fielmente as restrições e regras de negócio do contrato inteligente original.

3.4 Aplicação do Algoritmo

A aplicação prática do processo de transformação, descrito na Seção 3.3, é demonstrada a partir do código Solidity do sistema de restaurante. O objetivo é transformar o código Solidity da Listagem 1 na Rede de Petri Colorida da Figura 5.

```

1  contract Restaurante {
2      enum Cliente { C1, C2 }
3
4      enum Prato { Pizza, Pasta }
5
6      struct Comanda {
7          Cliente c;
8          Prato p;
9      }
10
11     mapping(Cliente => uint256) public clientesEsperando;
12     mapping(Prato => uint256) public pedidoCliente;
13     mapping(uint8 => mapping(uint8 => uint256)) public pedidosParaCozinha;
14     mapping(uint8 => mapping(uint8 => uint256)) public pratosNoBalcao;
15     mapping(Cliente => uint256) public clientesSatisfeitos;

```

```

16
17     event FazerPedidoDisparada(Cliente c, Prato p);
18     event CozinharDisparada(Cliente c, Prato p);
19     event EntregarPedidoDisparada(Cliente c, Prato p);
20
21     constructor() {
22         clientesEsperando[Cliente.C1] = 1;
23         clientesEsperando[Cliente.C2] = 1;
24         pedidoCliente[Prato.Pizza] = 1;
25     }
26
27     function fazerPedido(Cliente c, Prato p) external {
28         require(clientesEsperando[c] >= 1, "ClientesEsperando: marca ausente");
29         require(pedidoCliente[c] >= 1, "PedidoCliente: marca ausente");
30
31         clientesEsperando[c] -= 1;
32         pedidoCliente[c] -= 1;
33
34         pedidosParaCozinha[uint8(c)][uint8(p)] += 1;
35
36         emit FazerPedidoDisparada(c, p);
37     }
38
39     function cozinhar(Cliente c, Prato p) external {
40         require(
41             pedidosParaCozinha[uint8(c)][uint8(p)] >= 1,
42             "PedidosParaCozinha: comanda ausente"
43         );
44
45         pedidosParaCozinha[uint8(c)][uint8(p)] -= 1;
46         pratosNoBalcao[uint8(c)][uint8(p)] += 1;
47
48         emit CozinharDisparada(c, p);
49     }
50
51     function entregarPedido(Cliente c, Prato p) external {
52         require(
53             pratosNoBalcao[uint8(c)][uint8(p)] >= 1,
54             "PratosNoBalcao: prato ausente"
55         );
56
57         pratosNoBalcao[uint8(c)][uint8(p)] -= 1;
58         clientesSatisfeitos[c] += 1;
59
60         emit EntregarPedidoDisparada(c, p);
61     }
62 }

```

Listagem 1 – Código Solidity do Restaurante.

3.4.1 Análise o Contrato Solidity

A primeira etapa consiste em levantar as informações das tabelas auxiliares. As variáveis globais são descritas na Tabela 1, onde cinco registros são identificados (campos N, T, V, TI, VI). Cada lugar da RPC é obtido de uma variável de estado do contrato. Os

Tabela 1 – Tabela *Variáveis Globais* extraída do contrato Restaurante.

N	T	V	TI	VI
clientesEsperando	uint256	$1'C_1 ++ 1'C_2$	Cliente	public
pedidoCliente	uint256	$1'Pizza$	Prato	public
pedidosParaCozinha	uint256	“”	(uint8, uint8)	public
pratosNoBalcao	uint256	“”	(uint8, uint8)	public
clientesSatisfeitos	uint256	“”	Cliente	public

tipos enumerados *Cliente* e *Prato*, bem como a estrutura *struct Comanda*, não geram registros na Tabela 1 (Variáveis Globais) pois não representam dados armazenados no estado do contrato, porém esses tipos definem os domínios desses dados. No formalismo das RPCs, essas definições são mapeadas diretamente para o conjunto Σ de cores do modelo, estabelecendo os tipos de dados que os lugares estão autorizados a conter.

Cabe ressaltar que a geração de Conjuntos de Cores não é exclusiva para tipos customizados (*enums* ou *structs*). Em contratos que utilizam apenas tipos primitivos da Solidity, o processo de transformação mapeia tipos como *uint256*, *bool* ou *address* para conjuntos de cores baseados em inteiros, booleanos ou strings, respectivamente. O uso de *enums* no contrato do restaurante visa conferir maior expressividade semântica ao modelo formal, permitindo que a RPC capture a lógica de negócio da forma mais clara possível através dos tipos de dados complexos.

Por outro lado, estes tipos definem o conjunto Σ de cores do modelo, ou seja, $\Sigma = \{CLIENTE, PRATO, COMANDA\}$, com $COMANDA = CLIENTE \times PRATO$. Note que, embora em Solidity os mapeamentos usem *uint256* como contador de marcas, na RPC o que importa é o tipo do índice. Neste caso, a cor do lugar é determinada por $C(p)$. Assim, $C(clientesEsperando) = CLIENTE$, $C(pedidoCliente) = PRATO$, $C(pedidosParaCozinha) = C(pratosNoBalcao) = COMANDA$ e $C(clientesSatisfeitos) = CLIENTE$.

Na Tabela 2 quatro funções são identificadas usando os campos N , NT , T , V , NP , P , I , e F . A Tabela 3 determina as operações, através dos campos V , O , P , e L . Note

Tabela 2 – Tabela *Funções* extraída do contrato Restaurante.

N	T	V	NP	P	I–F
construtor	NULO	NULO	“”	“”	33–37
fazerPedido	“”	external	c, p	Cliente, Prato	49–60
cozinhar	“”	external	c, p	Cliente, Prato	67–76
entregarPedido	“”	external	c, p	Cliente, Prato	83–92

que as atribuições do construtor, os incrementos e decrementos sobre os mapeamentos são registrados na tabela. Por fim, na Tabela 4 as condicionais e restrições do código são apontadas com os campos T , C , I , e F . Os comandos *require* correspondem à condição de habilitação clássica de uma RPC.

Tabela 3 – Tabela *Operações* extraída do contrato **Restaurante**.

V	O	P	L	VI
clientesEsperando[C1]	= 1	$\emptyset$	34	public
clientesEsperando[C2]	= 1	$\emptyset$	35	public
pedidoCliente[Pizza]	= 1	$\emptyset$	36	public
clientesEsperando[c]	= 1	{c}	55	public
pedidoCliente[p]	= 1	{p}	56	public
pedidosParaCozinha[c] [p]	+ 1	{c, p}	58	public
pedidosParaCozinha[c] [p]	= 1	{c, p}	73	public
pratosNoBalcao[c] [p]	+ 1	{c, p}	74	public
pratosNoBalcao[c] [p]	= 1	{c, p}	89	public
clientesSatisfeitos[c]	+ 1	{c}	90	public

Tabela 4 – Tabela *Condicionais* extraída do contrato **Restaurante**.

T	C	I	F
require	clientesEsperando[c] >= 1	51	51
require	pedidoCliente[p] >= 1	52	52
require	pedidosParaCozinha[uint8(c)] [uint8(p)] >= 1	68	71
require	pratosNoBalcao[uint8(c)] [uint8(p)] >= 1	84	87

Já a tabela de chamadas permanece vazia, pois não há chamadas internas entre *fazerPedido*, *cozinhar* e *entregarPedido*. As três funções se comunicam exclusivamente por meio dos lugares (mapeamentos), refletindo de forma precisa a distinção entre as transições da RPC.

3.4.2 Obtendo a Rede de Petri Colorida

Na segunda etapa do processo, o Algoritmo 3 constrói a RPC que representa o contrato do **Restaurante** com base nas listas *VariáveisGlobais*, *Funções*, *Operações*, *Chamadas* e *Condicionais* produzidas na primeira etapa. A transformação é realizada conforme os passos descritos a seguir.

Passo 1: produção dos lugares

Na primeira parte, o Algoritmo 3 percorre a tabela *VariáveisGlobais* e gera, para cada variável, um lugar homônimo na rede. O campo *T* (combinado com *TI*, quando a variável é indexada) determina o conjunto de cores associado ao lugar, enquanto o campo *V* define a sua marcação inicial e *VI* sua visibilidade. Para o contrato do restaurante o algoritmo gera cinco lugares: *clientesEsperando* (cor *Cliente*), *pedidoCliente* (cor *Prato*), *pedidosParaCozinha* e *pratosNoBalcao* (cor *Comanda* = *Cliente* $\times$ *Prato*) e *clientesSatisfeitos* (cor *Cliente*). Como nenhuma das variáveis globais possui valor inicial atribuído na declaração, a marcação inicial *In* desses cinco lugares é vazia. Assim, todas as marcas passam a existir somente após a execução do construtor.

As linhas de 7–12 do algoritmo são responsáveis por criarem um lugar adicio-

nal para cada função com parâmetros, nomeado com o prefixo *par-* seguido do campo N da função. A cor desse lugar é o produto cartesiano dos tipos do campo P e do campo V , quando a função é *public* ou *external*. Este lugar é classificado como *oráculo*, indicando que suas marcas provêm do ambiente externo ao contrato. As três funções *fazerPedido*, *cozinhar* e *entregarPedido* são declaradas *external* e recebem o par (c, p) do tipo $\text{CLIENTE} \times \text{PRATO}$, dando origem aos lugares oráculo *par-fazerPedido*, *par-cozinhar* e *par-entregarPedido*, todos com marcação inicial vazia. O construtor não gera lugar do tipo *par-*, pois não possui parâmetros.

Essa fase de criação dos lugares para o contrato do restaurante gera oito lugares, onde três lugares são oráculos. Veja a Tabela 5.

Tabela 5 – Conjunto de lugares gerados para o contrato restaurante.

Lugar	Origem	Cor	In
<i>clientesEsperando</i>	var. global	CLIENTE	$\emptyset$
<i>pedidoCliente</i>	var. global	PRATO	$\emptyset$
<i>pedidosParaCozinha</i>	var. global	COMANDA	$\emptyset$
<i>pratosNoBalcao</i>	var. global	COMANDA	$\emptyset$
<i>clientesSatisfeitos</i>	var. global	CLIENTE	$\emptyset$
<i>par-fazerPedido</i>	parâm. (oráculo)	COMANDA	$\emptyset$
<i>par-cozinhar</i>	parâm. (oráculo)	COMANDA	$\emptyset$
<i>par-entregarPedido</i>	parâm. (oráculo)	COMANDA	$\emptyset$

Passo 2: produção das transições

As linhas de 13–16 do Algoritmo 3 tem a função de gerar as transições da RPC. O algoritmo percorre a Tabela 2 e gera, para cada entrada, uma transição com o mesmo nome da função. Como o Algoritmo 2 processa explicitamente o escopo de funções e construtores, o construtor do restaurante também deve constar como entrada da Tabela 2 e, consequentemente, dá origem a uma transição própria. Assim, o conjunto de transições da RPC para o restaurante é, portanto, composto por $\{\text{Constructor}, \text{FazerPedido}, \text{Cozinhar}, \text{EntregarPedido}\}$.

Passo 3: produção dos arcos

A construção dos arcos ocorre pela aplicação de três sub-regras (linhas de 17–32 do Algoritmo 3). A primeira (linhas de 18–19) determina que um arco deve ser criado, para uma função com parâmetros, do lugar *par-correspondente* para a transição da função. Assim, para o contrato do restaurante obtemos os arcos *par-fazerPedido* $\rightarrow$ *FazerPedido*, *par-cozinhar* $\rightarrow$ *Cozinhar* e *par-entregarPedido* $\rightarrow$ *EntregarPedido*.

A segunda sub-regra (linhas de 20–21) verifica se uma variável global é referenciada por alguma função, consultando as tabelas de *Operações* e *Condicionais*. Em caso

afirmativo, um arco é criado entre o lugar correspondente (variável) e a transição (função) e outro entre a transição e o lugar.

A modelagem com arcos em ambas as direções é necessária devido à semântica de disparo das Redes de Petri. Em uma Rede de Petri, qualquer acesso aos dados de um lugar exige o consumo de sua marca pelo arco de entrada, removendo a variável de estado do sistema. Para garantir a persistência e continuidade do contrato, a transição deve sempre produzir uma marca de volta para esse lugar por meio de um arco de saída. Por isso, a estrutura contendo dois arcos (um de entrada e um de saída) é aplicada de forma padrão para qualquer tipo de acesso ao estado, seja para uma simples leitura (onde a marca devolvida possui o mesmo valor da consumida) ou para uma escrita, tais como incrementos e decrementos (onde a marca devolvida carrega o valor atualizado).

Dessa forma, nove arcos duplos (dezoito arcos) são obtidos (formando nove pares em direções opostas) para o contrato Restaurante, onde o símbolo $\leftrightarrow$ representa a existência de dois arcos em direções opostas:

```

Constructor $\leftrightarrow$ clientesEsperando,
Constructor $\leftrightarrow$ pedidoCliente,
FazerPedido $\leftrightarrow$ clientesEsperando,
FazerPedido $\leftrightarrow$ pedidoCliente,
FazerPedido $\leftrightarrow$ pedidosParaCozinha,
Cozinhar $\leftrightarrow$ pedidosParaCozinha,
Cozinhar $\leftrightarrow$ pratosNoBalcao,
EntregarPedido $\leftrightarrow$ pratosNoBalcao e
EntregarPedido $\leftrightarrow$ clientesSatisfeitos.

```

A terceira sub-regra (linhas de 26 a 31) trata das chamadas de subfunções. O algoritmo percorre a tabela de *Chamadas* e, para cada registro, identifica a transição correspondente à função que realizou a invocação (transição chamadora) e o lugar associado aos parâmetros da função invocada (lugar de destino). Em seguida, um arco direcional é criado da transição chamadora para o lugar de destino, modelando formalmente a passagem de argumentos e a transferência do fluxo de execução entre as rotinas do contrato.

No contrato do Restaurante as funções não realizam chamadas internas entre si e a tabela de Chamadas está vazia. Logo, nenhum arco precisa ser gerado nesta etapa. Somando-se os três arcos dos lugares oráculo aos dezoito arcos, a RPC resultante possui um total de vinte e um arcos.

Passo 4: expressões de arco e guardas

Por fim, o Algoritmo 3 (linhas de 33–42) associa as expressões aos arcos de entrada e saída. Para os arcos de entrada, o objetivo é realizar a amarração dos valores

relação com a transformação?

A ver inclusive esses arcos duplos. A figura 5 não tem. Logo, não faz a mesma coisa?

MSN: corrigido, explicado a baixo também a questão das expressões

Tabela 6 – Comparativo entre a RPC da Figura 5 e a RPC obtida.

Elemento	Figura 5	R_{lit}
Lugares	5	8
Oráculos	0	3
Transições	3	4
Arcos	9	21
In não-vazia	sim	não
Guardas $G(t)$ não triviais	não	sim

Note que a RPC R não coincide de forma literal com a rede da Figura 5, pois o contrato original em Solidity possui verbosidades não presentes na rede de Petri Colorida. O construtor juntamente com a necessidade de se criar lugares oráculos, conforme os parâmetros de funções, resulta em uma rede mais complexa, porém, com a mesma funcionalidade da RPC da Figura 5. Outro aspecto fundamental da transformação é a interpretação dos comandos de validação, como o *require*, que são modelados como expressões de guarda nas transições, empregando o mecanismo nativo de restrição para disparos das RPCs. Esse mecanismo garante que as lógicas de negócio do contrato sejam rigorosamente respeitadas na rede, tornando a rede mais verbosa e com uma sintaxe diferente de uma rede convencional.

Para mostrar a correspondência entre uma RPC obtida de forma convencional e uma resultante da transformação proposta a partir de um código Solidity é necessário demonstrar que ambas preservam a mesma semântica de execução. Essa correspondência é verificada se, sob as mesmas entradas operacionais, ambas as topologias através dos mecanismos de amarração (*binding*), habilitação e disparo, evoluem com o consumo e produção das mesmas marcações.

Observe que a principal diferença estrutural para a RPC R está nas restrições arquiteturais da EVM, que utiliza lugares oráculo para representar as invocações externas, e pares de arcos em direções opostas para simular a leitura e escrita de variáveis de estado persistentes e expressões de guarda explícitas mapeadas a partir das condicionais do código Solidity.

A seguir é apresentada a execução comparativa do fluxo completo (inicialização, pedido, preparo e entrega) para um cenário entre as duas RPCs, onde o cliente $C1$ solicita uma *Pizza*:

1. Estado Inicial e a Transição do Construtor

- **Rede Original:** Assume uma marcação inicial já distribuída de forma estática, onde a função *In* atribui $1'C1 + +1'C2$ ao lugar *ClientesEsperando* e $1'Pizza$ a *PedidoCliente*.
- **Rede Gerada (*R*):** A rede inicia vazia. A equivalência do estado inicial é assegurada pela transição *Constructor*. Como sua expressão de guarda é trivial ($G : true$), então a transição está habilitada e o seu disparo produz as marcas iniciais mapeadas, alocando $1'C1 + +1'C2$ na variável global *clientesEsperando* e $1'Pizza$ em *pedidoCliente*. O Construtor é, portanto, o responsável por preencher os valores iniciais do contrato. Após esse disparo, as marcações de ambas as redes são idênticas nos estados correspondentes.

2. Execução da Transição “Fazer Pedido”

- **Amarração:** Na rede original, a amarração válida para as variáveis do arco é $b = \langle c = C1, p = Pizza \rangle$. Na RPC *R*, o processo é iniciado com o gatilho da invocação externa, produzindo uma marca $(C1, Pizza)$ no lugar oráculo *par-fazerPedido*, mantendo uma amarração de arco igual a *b*.
- **Habilitação:** A rede original habilita a transição pela disponibilidade das marcas nos lugares adjacentes. Em contrapartida, *R* possui um guarda $G : clientes[c] \geq 1 \wedge pedidoCliente[p] \geq 1$, mapeado a partir das restrições (*require*) do contrato que precisa ser satisfeito.

A transição é habilitada pela presença da marca no lugar oráculo em conjunto com a disponibilidade das marcas referentes ao estado persistente nos lugares *clientesEsperando* e *pedidoCliente*. Como o Construtor já havia alocado as marcas correspondentes ao estado, as expressões dos arcos de entrada e a guarda resultam em verdadeiro, habilitando a transição. Caso não exista tais marcas nos lugares de estado, a transição não habilita.

- **Disparo:** Ao disparar, a rede original consome as marcas *C1* do lugar *ClientesEsperando* e *Pizza* do lugar *PedidoCliente*, produzindo $1'(C1, Pizza)$ no lugar *Pedidos p/ Cozinha*. Em *R_{lit}* a transição consome a marca de invocação do oráculo e, através dos arcos de entrada, consome as marcas correspondentes aos estados nos lugares das variáveis globais. Através dos arcos de saída, a transição produz as marcas atualizadas de volta para esses lugares de estado, simulando a subtração e atualização do mapeamento no Solidity, e gerando a marca $(C1, Pizza)$ no lugar *pedidosParaCozinha*.

3. Execução da Ação “Cozinhar”

- **Amarração:** A cozinha deve processar a comanda. Na rede original a amarração válida ocorre pela presença da marca da comanda no lugar *Pedidos p/*

Cozinha, resultando em $b = \langle c = C1, p = Pizza \rangle$. Em R_{lit} . O processo se inicia com a invocação externa (o gatilho da ação) que insere a marca $(C1, Pizza)$ no lugar oráculo *par-cozinhar*, resultando na mesma amarração b .

- **Habilitação:** A rede original habilita a transição implicitamente pela disponibilidade da comanda. Em contrapartida, R_{lit} avalia a guarda $G : pedidosParaCozinha[c, p] \geq 1$.
1. A transição habilita se, através dos arcos de leitura, é confirmada a disponibilidade efetiva da marca correspondente ao pedido no lugar de estado *pedidosParaCozinha* (representando o estado persistente do contrato). Como essa marca é produzida e alocada na etapa anterior, a avaliação resulta em verdadeiro e a transição é habilitada.
- **Disparo:** A rede original consome a marca de entrada e a transfere para *Pratos no Balcão*. Analogamente, a rede gerada consome a requisição do oráculo, produzindo a marca correspondente $(C1, Pizza)$ no lugar de estado estrutural *pratosNoBalcao*.

4. Execução da Ação “Entregar Pedido”

- **Amarração:** Na etapa final, a rede original alcança uma amarração válida pela presença da marca correspondente ao prato no lugar *Pratos no Balcão*, resultando na amarração $b = \langle c = C1, p = Pizza \rangle$. Em R_{lit} o processo inicia com a invocação externa da função *entregarPedido* (o gatilho da ação), que insere a marca de *Comanda* $(C1, Pizza)$ no lugar oráculo *par-entregar*, resultando na mesma amarração b .
- **Habilitação:** A rede original habilita a transição implicitamente pela disponibilidade da marca do prato no balcão. Em contrapartida, R_{lit} avalia a guarda $G : pratosNoBalcao[c, p] \geq 1$ e a transição é habilitada se, através dos arcos de leitura, é confirmada a disponibilidade da marca correspondente no lugar de estado *pratosNoBalcao* (representando o estado persistente do contrato). Como essa marca é disponibilizada na etapa anterior, a avaliação do guarda resulta em verdadeiro e a transição é habilitada.
- **Disparo e Estado Final:** O disparo na rede original retira a comanda do balcão e produz a marca do cliente $C1$ no lugar final *Clientes Satisfeitos*. Em R_{lit} o comportamento é matematicamente idêntico: a transição é executada consumindo a requisição do oráculo e a variável global *clientesSatisfeitos* recebe a marca referente a $C1$.

ALB: Estes [2] são citação? Vieram de algum copia/cola? MSN: corrigido

Ao final da execução das redes, constata-se que ambas evoluem para o mesmo estado final. O processamento lógico da comanda é concluído com sucesso e o cliente

$C1$ atinge o lugar/variável de destino. Dessa forma, pode-se notar que a adição de complexidade estrutural na rede gerada (verbosidade advinda de oráculos e persistência de dados) serve exclusivamente para adequar o modelo aos contratos inteligentes, sem alterar, corromper ou desviar a semântica de execução fundamental da Rede de Petri Colorida original.

4 APLICAÇÃO PRÁTICA

Devido a grande importância da atividade de verificação sobre contratos, eletrônicos ou inteligentes, já estabelecida anteriormente, para garantir corretude, este capítulo tem o objetivo de apresentar um teste de conceito usando através da verificação de um contrato real de compra e venda.

Algumas técnicas de verificação, como a verificação baseada em RCL, permite uma análise rigorosa das cláusulas que descrevem o contrato, além da detecção de conflitos normativos em contratos multilaterais [3]. Como um dos objetivos do trabalho é estender a verificação mais rigorosa para contratos inteligentes, pretende-se aplicar a técnica formal de verificação de contratos eletrônicos usando a RCL para o caso de contratos inteligentes.

Para o teste de conceito o contrato de compra e venda, usado como estudo de caso em [3], é considerado como ponto de partida. Primeiramente, o contrato e suas regras, expressas em RCL, serão apresentados, inicialmente, com um conflito. Na sequência o contrato é analisado, verificado com a ferramenta. corrigido e, posteriormente, verificado novamente. Após isso, o mesmo contrato é implementado em Solidity, a partir do RCL original, especificando as obrigações, permissões e proibições do modelo formal para uma programa executável. Por fim, o contrato é testado a fim de garantir que a semântica do contrato escrito em RCL seja traduzida para o *smart contract*, e em seguida, que uma segunda versão, agora baseada na RCL corrigida, seja testada novamente.

Este estudo possibilita uma prova de conceito sobre a transformação entre contratos escritos em Solidity e suas respectivas descrições em RCL, além de permitir uma análise sobre os desafios e as potenciais perdas de informação que ocorrem nessa tradução, reforçando a necessidade de uma abordagem sistemática para garantir a corretude e a segurança sobre os contratos inteligentes.

4.1 Um Contrato em RCL

Um contrato de compra e venda, apresentado em [3], descreve um cenário de compra digital composta por diferentes agentes: um comprador, que realiza a aquisição de produtos; um vendedor que oferece tais produtos; uma transportadora também faz parte do processo, com a tarefa de entregar o produto ao comprador; e um banco intermedeia as transações financeiras. Além do acordo entre as partes, também são consideradas regras internas dos envolvidos. Neste cenário, a transportadora define uma regra interna segundo a qual um produto só deve ser entregue se o frete já tiver sido pago pelo vendedor. Outra regra interna é estabelecida pelo banco, que proíbe a realização de pagamentos quando da ausência de notificações requeridas para evitar fraudes.

A descrição do contrato em linguagem natural é dada como segue:

1. O **Comprador** realiza a compra de um produto do **Vendedor**.
2. O **Comprador** é obrigado a pagar o produto ao **Banco**.
3. O **Banco** deve enviar a notificação sobre o pagamento do produto ao **Vendedor**
4. Após o **Banco** notificar o **Vendedor** sobre o pagamento, o **Vendedor** é obrigado a enviar o produto por meio de um **Transportador** e pagar os custos de envio do produto ao **Banco**.
5. O **Transportador** deve entregar o produto ao **Comprador**.
6. Após a entrega do produto, o **Comprador** é obrigado a informar o **Banco** sobre a entrega do produto, enquanto o **Transportador** deve notificar o **Vendedor** que o produto foi entregue ao **Comprador**.
7. Quando o **Vendedor** é notificado sobre a entrega do produto, o **Vendedor** é obrigado a notificar o **Banco** para liberar o pagamento dos custos de envio ao **Transportador**.
8. Quando o **Comprador** notifica o **Banco** sobre a entrega do produto, o **Banco** libera o pagamento correspondente ao **Vendedor**].
9. O **Banco** deve pagar os custos de envio ao **Transportador** após o **Vendedor** efetuar o pagamento do valor referido.

Regras Internas do Banco

10. O **Banco** está proibido de pagar o **Vendedor** até que uma notificação adequada seja recebida do **Comprador** confirmando que o produto foi entregue.
11. O **Banco** está proibido de liberar o pagamento dos custos de envio para o **Transportador** até que o **Vendedor** notifique o banco.

Regras Internas do transportador

12. O **Transportador** está proibido de entregar o produto até que o **Vendedor** tenha pago os custos de envio.

Com o intuito de facilitar a compreensão do contrato abordado, a Tabela 7 apresenta as ações do contrato e suas respectivas descrições. Conforme abordado em [3], a

ação	descrição
buyProduct	Comprar o produto
payProduct	Pagar o valor do produto
notifyProductPayment	Notificar o pagamento do produto
sendProduct	Enviar o produto
deliverProduct	Entregar o produto
notifyProductReceipt	Notificar o recebimento do produto
notifyProductDelivery	Notificar a entrega do produto
payShippingCosts	Pagar o frete
releaseShippingCosts	Liberar o pagamento do frete

Tabela 7 – Ações do contrato

descrição do contrato em RCL é obtida a partir das regras e restrições apresentadas em linguagem natural. A Figura 7 mostra o contrato de compra e venda em RCL. O contrato

```

1  {b, s}[buyProduct](
2    {b, k}O(payProduct)^
3    {b, k}[payProduct](
4      {k, s}O(notifyProductPayment)^
5      {k, s}[notifyProductPayment](
6        {s, c}O(sendProduct)^
7        {s, k}O(payShippingCosts)^
8        {s, k}[payShippingCosts](
9          {s, c}[sendProduct](
10           {c, b}O(deliverProduct)^
11           {c, b}[deliverProduct](
12             {b, k}O(notifyProductReceipt)^
13             {c, s}O(notifyProductDelivery)
14             {b, k}[notifyProductReceipt]({k, s}O(payProduct))^
15             {c, s}[notifyProductDelivery](
16               {s, k}O(liberateShippingCosts)^
17               {s, k}[liberateShippingCosts]
18               ({k, c}O(payShippingCosts))
19             )))))));
20 {b, k}[¬notifyProductReceipt] * ({k, s}F(payProduct));
21 {s, k}[¬liberateShippingCosts] * ({k, c}F(payShippingCosts));
22 {s, c}[¬payShippingCosts] * ({c, b}F(deliverProduct));

```

Figura 7 – Contrato de compra e venda em RCL

em RCL é composto por quatro participantes: buyer (b), o comprador; seller (s), o vendedor; bank (k), o intermediário para transações financeiras; e o carrier (c), responsável pelo transporte do produto. Cada ação do contrato é caracterizada pela relativização do tipo um para um, onde em cada ação as partes são especificadas. Por exemplo, a ação *buyProduct* só pode ser realizada do buyer para o seller, nessa ordem. Outra característica do contrato é a presença do operador deôntico de obrigação e a penalidade associada, caso uma violação ocorra. Por fim, as regras internas do banco e transportadora também são especificadas.

No trabalho original [3], o contrato de compra e venda foi verificado através da ferramenta RECALL [33], desenvolvida com o objetivo de analisar contratos multilaterais expressos em RCL. A verificação desse contrato pela ferramenta RECALL [3] apontou a presença de um conflito na cláusula de $\{c, b\}(deliverProduct)$ com $\{c, b\}O(deliverProduct)$. A análise realizada pela ferramenta RECALL apontou que a regra interna da transportadora estava em conflito com o restante do contrato.

A regra interna estabelecia que a transportadora poderia enviar o produto somente após o pagamento do frete. Por outro lado, no contrato original, espera-se que o produto seja entregue pela transportadora ao comprador antes que o pagamento do frete seja liberado pelo banco. Portanto, existe uma incompatibilidade na regra interna da transportadora, pois o pagamento do frete, realizado pelo vendedor, era esperado antes do envio do produto ao comprador. Além disso, a transportadora também considerava, em sua regra interna, que o pagamento do frete deveria ser realizado pelo vendedor, e não pelo banco, para então proceder à entrega do produto.

Para a correção desse conflito, uma nova cláusula foi incluída no contrato, indicando que o banco então deveria notificar a transportadora sobre o pagamento do frete pelo vendedor. Outra alteração foi realizada especificamente na regra interna da transportadora, que agora deveria levar em consideração a notificação do banco como garantia de pagamento do frete.

O contrato revisado com a correção dos conflitos altera a cláusula 5 e as regras internas da transportadora. As cláusulas modificadas do contrato original são apresentadas a seguir:

[Regra do contrato de compra e venda]

5. O **Banco** deve notificar a **Transportadora** sobre o pagamento do frete e após o **Banco** atestar o pagamento, a **Transportadora** é obrigada a entregar o produto para o **Comprador**.

[Regra interna da Transportadora]

12. A **Transportadora** é proibida de entregar o produto até que o **Banco** notifique-a de que o **Vendedor** pagou o valor do frete.

Após essas correções, o contrato em RCL modificado é descrito na Figura 8. Este novo contrato foi submetido novamente à ferramenta RECALL e a análise obtida retornou que o contrato estava livre de conflitos.

Ferramentas como a RECALL são de suma importância para assegurar contratos livres de conflitos e garantir sua corretude. Um caminho para a verificação de contratos inteligentes é o uso de ferramentas similares ao RECALL. No entanto, a ferramenta RECALL lida com contratos eletrônicos escritos em RCL, não sendo diretamente aplicados a cláusulas contratuais expressas em linguagens de programação, como é o caso da Solidity. Uma das linhas de trabalho procura sistematizar a tradução de contratos inteligentes para formalismos adequados, como as Redes de Petri, para que estes contratos possam então ser especificados formalmente e verificados por alguma ferramenta automática de suporte.

```

1  {b, s}[buyProduct](
2    {b, k}O(payProduct)∧
3    {b, k}[payProduct](
4      {k, s}O(notifyProductPayment)∧
5      {k, s}[notifyProductPayment](
6        {s, c}O(sendProduct)∧
7        {s, k}O(payShippingCosts)∧
8        {s, k}[payShippingCosts](
9          {k, c}O(notifyShippingPayment) ∧ {s, c}[sendProduct](
10           {c, b}O(deliverProduct)∧
11           {c, b}[deliverProduct](
12             {b, k}O(notifyProductReceipt)∧
13             {c, s}O(notifyProductDelivery)∧
14             {b, k}[notifyProductReceipt]({k, c}O(payProduct))∧
15             {c, s}[notifyProductDelivery](
16               {s, k}O(liberateShippingCosts)∧
17               {s, k}[liberateShippingCosts]({k, c}O(payShippingCosts))
18             )))))));
19 {b, k}[(notifyProductReceipt)*]({k, s}F(payProduct));
20 {s, k}[(liberateShippingCosts)*]({k, c}F(payShippingCosts));
21 {k, c}[(notifyShippingPayment)*]({c, b}F(deliverProduct));

```

Figura 8 – Contrato de compra e venda corrigido em RCL

4.2 Um Contrato Inteligente em Solidity

Nesta seção, o contrato de compra e venda abordado na Seção 4.1 é implementado em Solidity. A obtenção do contrato inteligente que implementa a compra e venda de produtos tal como descrito anteriormente foi concebido com base nas cláusulas em RCL. Essas regras descritas nas cláusulas do contrato são implementadas como funções em Solidity. O processo de obtenção do contrato em Solidity a partir da RCL é o mesmo adotado tanto para o contrato onde existe um conflito quanto para o contrato corrigido.

Esse método apresenta alguns riscos a validade do processo de verificação, já que a transformação não é sistemática e rigorosa para garantir uma representação equivalente. Neste sentido, caso haja qualquer erro na interpretação dada ao contrato em RCL, este pode ser implementado na sua versão em Solidity.

4.2.1 Implementação em Solidity

Esta seção apresenta o código Solidity para o primeiro contrato de compra e venda descrito na Seção 4.1. Esse código é uma tradução do RCL que contém um erro para

Solidity. Logo, espera-se que sua execução apresente um erro assim como encontrado no contrato RCL.

Logo, o contrato em Solidity também possui quatro agentes: o comprador, o vendedor, o banco e a transportadora; identificados pelos endereços de suas contas no Ethereum. Além disso, o contrato possui estados que modelam seu fluxo de controle, já que na RCL as cláusulas possuem uma ordem correta de execução. Essa característica é implementada no contrato inteligente para que suas funções não ocorram de forma desordenada, acarretando em inconsistências não presentes na RCL. Os estados de controle criados no contrato inteligente são *Created*, *ProductBought*, *ProductPaid*, *PaymentNotified*, *ProductDelivered* e *Finalized*, e representam, respectivamente, a criação do contrato, a compra de um produto, o pagamento do produto realizado pelo comprador, a notificação de pagamento da compra realizada, a entrega do produto, e a finalização do contrato.

A passagem do controle de um estado para outro indica as mudanças e ações que representam as cláusulas do contrato, bem como as regras internas do banco e da transportadora. Algumas variáveis também servem para controlar as ações concorrentes do contrato, como por exemplo, as ações realizada após o pagamento do produto ser notificado: (a) enviar o produto; e (b) pagar o frete. Após essas duas ações, especificadas por duas variáveis booleanas, serem concluídas, a próxima ação pode então ser executada.

Outras variáveis de controle cumprem o mesmo papel no contrato: *receiptNotifiedByBuyer*, *deliveryNotifiedByCarrier*, *paymentReleasedSeller*, *paymentReleasedCarrier*. Já a regra interna da transportadora é traduzida para a variável booleana *paymentReleasedCarrierSeller*, enquanto que a regra interna do banco é especificada pela variável *receiptNotifiedByBuyer*, que muda seu valor verdade quando o comprador notifica o recebimento do produto para que o banco possa liberar o valor da venda para o vendedor, e pela função *liberateShippingCosts*, que realiza o pagamento do frete para a transportadora caso nenhum erro ocorra.

O código também define os eventos que notificam as mudanças de estado e os modificadores que verificam o agente responsável pela ação: *onlyB*, *onlyS*, *onlyC* e *onlyK*. O modificador *onlyB*, por exemplo, verifica o responsável pela ação do agente que está realizando a compra através do seu endereço. Caso não seja do comprador, a função não é realizada, garantindo a característica da relativização modelada pela RCL, onde cada ação tem um participante responsável associado. O modificador pode ser observado na função *buyProduct*, por exemplo, onde apenas o usuário que possui o endereço específico de comprador tem a permissão de realizar a compra do produto.

O construtor do contrato, uma função especial executada apenas uma vez na implantação do contrato na *Blockchain*, é o responsável por receber informações ou instruções que inicializam o contrato, bem como o construtor de uma classe. Nesse caso, o contrato recebe as informações das contas dos participantes e dos valores de frete e produto, por

meio de seis parâmetros:

- os endereços Ethereum (*address*) dos quatro participantes: `_buyer`, `_seller`, `_bank` e `_carrier`;
- o valor total do pagamento (`_paymentAmount`);
- o valor do frete.

Estas informações são armazenadas nas variáveis de estado do contrato declaradas no início do código e permanecem imutáveis durante o ciclo de vida do contrato. O construtor também inicializa o estado do contrato como *Created*, marcando o início do processo de compra e venda. Já as funções especificam as cláusulas do contrato, dispostas na mesma ordem em que aparecem no contrato descrito anteriormente. Os modificadores garantem que as funções sejam executadas somente mediante certas condições. Os modificadores *only* garantem que certas ações só podem ser executadas pelas suas respectivas partes. Por exemplo, o modificador *onlyB* é onde apenas o *buyer* pode executar. Já os modificadores de *internalRules* são usados para descrever as regras internas descritas no contrato.

O contrato de compra e venda escrito em Solidity é apresentado na Listagem 1.

```

1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.0;
3
4 contract ContratoComErro {
5     address public buyer;
6     address public seller;
7     address public bank;
8     address public carrier;
9     uint public paymentAmount;
10    uint public shippingCosts;
11
12    enum ContractState {
13        Created,
14        ProductBought,
15        ProductPaid,
16        PaymentNotified,
17        ProductDelivered,
18        Finalized
19    }
20    ContractState public state;
21
22    bool private productSent = false; // {s,c}0(sendProduct)
23    bool private shippingCostsPaid = false; // {s,k}0(payShippingCosts)
24    bool private receiptNotifiedByBuyer = false; // {b,k}0(notifyProductReceipt)
25    bool private deliveryNotifiedByCarrier = false; // {c,s}0(
        notifyProductDelivery)
26    bool private paymentReleasedSeller = false;
27    bool private paymentReleasedCarrier = false;
28    bool private paymentRealeasedCarrierSeller = false;

```

```

29
30     event Notify(
31         address indexed sender,
32         address indexed receiver,
33         string message
34     );
35
36     modifier onlyB() {
37         require(msg.sender == buyer, "Apenas o Comprador (b)");
38         _;
39     }
40     modifier onlyS() {
41         require(msg.sender == seller, "Apenas o Vendedor (s)");
42         _;
43     }
44     modifier onlyK() {
45         require(msg.sender == bank, "Apenas o Banco (k)");
46         _;
47     }
48     modifier onlyC() {
49         require(msg.sender == carrier, "Apenas a Transportadora (c)");
50         _;
51     }
52
53     constructor(
54         address _buyer,
55         address _seller,
56         address _bank,
57         address _carrier,
58         uint _paymentAmount,
59         uint _shippingCosts
60     ) {
61         buyer = _buyer;
62         seller = _seller;
63         bank = _bank;
64         carrier = _carrier;
65         paymentAmount = _paymentAmount;
66         shippingCosts = _shippingCosts;
67         state = ContractState.Created;
68     }
69
70     modifier atState(ContractState _requiredState) {
71         require(state == _requiredState, "Estado invalido para essa acao");
72         _;
73     }
74
75     // 1. {b,s}[buyProduct](...)
76     function buyProduct() external onlyB atState(ContractState.Created) {
77         state = ContractState.ProductBought;
78         emit Notify(buyer, seller, "1. Comprador realizou a compra.");
79     }
80
81     // 2. {b,k}0(payProduct)
82     function payProduct()
83     external
84     payable
85     onlyB

```

```

86  atState(ContractState.ProductBought)
87  {
88      require(msg.value == paymentAmount, "Valor do pagamento incorreto");
89      state = ContractState.ProductPaid;
90      emit Notify(buyer, bank, "2. Comprador pagou o produto ao banco.");
91  }
92
93  // 4. {k,s}0(notifyProductPayment)
94  function notifyProductPayment()
95  external
96  onlyK
97  atState(ContractState.ProductPaid)
98  {
99      state = ContractState.PaymentNotified;
100     emit Notify(
101         bank,
102         seller,
103         "4. Banco notificou o vendedor sobre o pagamento."
104     );
105 }
106
107 // 6. {s,c}0(sendProduct)
108 function sendProduct()
109 external
110 onlyS
111 atState(ContractState.PaymentNotified)
112 {
113     require(!productSent, "Produto ja foi enviado");
114     productSent = true;
115     emit Notify(
116         seller,
117         carrier,
118         "6. Vendedor enviou o produto para a transportadora."
119     );
120 }
121
122 // 7. {s,k}0(payShippingCosts)
123 function payShippingCosts()
124 external
125 payable
126 onlyS
127 atState(ContractState.PaymentNotified)
128 {
129     require(msg.value == shippingCosts, "Valor do frete incorreto");
130     require(!shippingCostsPaid, "Frete ja foi pago");
131     shippingCostsPaid = true;
132     emit Notify(seller, bank, "7. Vendedor pagou o frete ao banco.");
133 }
134
135 // 10. {c,b}0(deliverProduct)
136 // 21. {s,c}[(!payShippingCosts)*]({c,b}F(deliverProduct))
137 function deliverProduct() external onlyC {
138     require(productSent, "Produto ainda nao foi enviado pelo vendedor");
139
140     require(
141         shippingCostsPaid,
142         "ERRO : Transportadora nao pode entregar antes do frete ser pago."

```



```

143     );
144
145     require(
146     paymentRealeasedCarrierSeller,
147     "Frete nao foi pago pelo vendedor a transportadora"
148     );
149
150     state = ContractState.ProductDelivered;
151     emit Notify(
152     carrier,
153     buyer,
154     "10. Transportadora entregou o produto ao comprador."
155     );
156 }
157
158 // 12. {b,k}0(notifyProductReceipt)
159 function notifyProductReceipt()
160 external
161 onlyB
162 atState(ContractState.ProductDelivered)
163 {
164     require(!receiptNotifiedByBuyer, "Recebimento ja foi notificado");
165     receiptNotifiedByBuyer = true;
166     emit Notify(
167     buyer,
168     bank,
169     "12. Comprador notificou o banco do recebimento."
170     );
171 }
172
173 // 13. {c,s}0(notifyProductDelivery)
174 function notifyProductDelivery()
175 external
176 onlyC
177 atState(ContractState.ProductDelivered)
178 {
179     require(!deliveryNotifiedByCarrier, "Entrega ja foi notificada");
180     deliveryNotifiedByCarrier = true;
181     emit Notify(
182     carrier,
183     seller,
184     "13. Transportadora notificou o vendedor da entrega."
185     );
186 }
187
188 // 14. {b,k}[notifyProductReceipt]({k,s}0(payProduct))
189 // 19. Regra Interna do Banco
190 function payProductSeller()
191 external
192 onlyK
193 atState(ContractState.ProductDelivered)
194 {
195     require(
196     receiptNotifiedByBuyer,
197     "Regra Interna B: Comprador ainda nao confirmou o recebimento."
198     );
199

```

```

200     emit Notify(bank, seller, "14. Banco liberou o pagamento ao vendedor.");
201     paymentReleasedSeller = true;
202     checkFinalization();
203 }
204
205 // 16. {s,k}0(liberateShippingCosts)
206 function liberateShippingCosts()
207 external
208 onlyS
209 atState(ContractState.ProductDelivered)
210 {
211     require(
212         deliveryNotifiedByCarrier,
213         "Vendedor ainda nao foi notificado pela transportadora."
214     );
215
216     emit Notify(
217         seller,
218         bank,
219         "16. Vendedor autorizou banco a liberar frete."
220     );
221     payShippingCostsToCarrier();
222 }
223
224 // 17. {k,c}0(payShippingCosts)
225 // 20. Regra Interna do Banco
226 function payShippingCostsToCarrier() private {
227     emit Notify(bank, carrier, "17. Banco pagou o frete a transportadora.");
228     paymentReleasedCarrier = true;
229     checkFinalization();
230 }
231
232 function checkFinalization() private {
233     if (state == ContractState.ProductDelivered) {
234         if (paymentReleasedCarrier && paymentReleasedSeller) {
235             state = ContractState.Finalized;
236         }
237     }
238 }
239 }

```

Listagem 1 – Código solidity do contrato com conflito.

ALB: Esse paragrafro abaixo nao é sobre RCL e RECALL apenas? Nao deveria estar apenas na secao 4.1, ali para o final? MSN: tem razão, ele ficou meio repetido na verdade, devo retirar esse paragrafo?

ALB: Sim, ou levar para lá, caso não esteja claro e com esses detalhes. O conflito existente no contrato original em RCL foi detectado pela ferramenta RECALL, conforme descrito na Seção 4.1. Em sua análise, a RECALL mostrou que a cláusula $F(c, deliverProduct, b)$ estava em conflito com $O(c, deliverProduct, b)$. A primeira cláusula provém da regra interna da transportadora, enquanto que a segunda, tem relação com a descrição geral do contrato. A regra interna estabelece que a transportadora pode enviar o produto somente após o pagamento do frete. Por outro lado, no contrato original, espera-se que o produto seja

entregue ao comprador, pela transportadora, antes que o pagamento do frete seja liberado pelo banco. Portanto, uma incompatibilidade ocorre na regra interna da transportadora, onde o pagamento do frete realizado pelo vendedor era esperado pela transportadora antes que o produto fosse enviado ao comprador. Já a transportadora, por sua regra interna, esperava que o pagamento do frete fosse realizado pelo vendedor, ao invés do banco, para que então o produto pudesse ser entregue.

Esse contrato em Solidity obtido da RCL original foi implantado na plataforma Remix IDE. A execução do contrato na plataforma se deu da seguinte forma:

1. o contrato é criado com os respectivos endereços de cada participante, com um valor para o produto a venda (um inteiro qualquer) e valor associado ao frete;
2. a função *buyProduct* é executada pela conta do buyer, mudando o estado do contrato de *Created* para *ProductBought*;
3. após o compra do produto, de forma sequencial, a função *payProduct* também é executada pelo buyer e muda o estado do contrato de *Created* para *ProductPaid*;
4. na sequência, o banco notifica o pagamento através da função *notifyProductPayment*. Se a função é executada com sucesso, o controle da execução passa para o estado *PaymentNotified*.
5. o seller então executa duas funções, *sendProduct* e *payShippingCosts*. Se as duas funções são executadas com sucesso, o estado do contrato é alterado para *PaymentNotified*;
6. em seguida, a função *deliverProduct* é executada e a mensagem de erro “Frete não foi pago pelo vendedor a transportadora” é emitida, como esperado, já que essa é uma regra interna da transportadora. Essa regra impede a execução correta do contrato, pois há um conflito entre as cláusulas: o produto deve ser enviado, para depois entregar o produto; e o pagamento deve ser realizado para que depois o produto seja enviado. Porém só uma delas é satisfeita neste ponto da execução (o frete foi pago). Portanto, a função não é executada de forma correta, passando o contrato para o próximo estado, interrompendo a execução do contrato.

4.2.1.1 Versão Corrigida em Solidity

Assim como a versão original em RCL foi transformada para código Solidity, a versão corrigida em RCL, apresentada na Seção 4.1, também foi descrita para um código Solidity, usando o mesmo método de interpretação aplicado na primeira transformação. A modificação, como observado, consiste então em o banco notificar a transportadora quando o pagamento do frete é efetuado pelo vendedor. Dessa forma, a transportadora

considera essa notificação do banco como garantia para liberar a entrega do produto ao comprador.

O contrato em Solidity resultante da transformação possui então algumas diferenças da primeira versão, como era de se esperar. A variável *paymentReleasedCarrierSeller*, que modelava a antiga regra da transportadora, foi alterada para *shippingPaymentNotified*. A mudança do nome da variável, claramente, não resolve o conflito em si, mas mantém um alinhamento com a nova regra da transportadora. O conflito, propriamente dito, é resolvido com a adição de uma nova função, chamada *notifyShippingPaymentToCarrier*, que é executada antes do produto ser enviado para o comprador. Essa função reflete a descrição da linha 9 do contrato RCL corrigido, onde após o vendedor pagar a taxa do frete para o banco, esse deve informar a transportadora da realização do pagamento (em RCL é a cláusula *notifyShippingPayment*), para que a transportadora possa enviar o produto. Uma última modificação é realizada na função *deliverProduct*, onde a variável *paymentReleasedCarrierSeller* gerava um impasse e impedia o contrato de evoluir. A nova função *deliverProduct* substitui a regra *paymentReleasedCarrierSeller* para *shippingPaymentNotified*, resolvendo o problema. As alterações propostas estão descritas no trecho de código na Listagem 2.

```

1
2  function notifyShippingPaymentToCarrier()
3  external
4  onlyK
5  atState(ContractState.PaymentNotified)
6  {
7      require(
8          shippingCostsPaid,
9          "O vendedor ainda nao pagou o frete ao banco."
10     );
11     require(
12         !shippingPaymentNotified,
13         "Notificacao de frete ja foi enviada."
14     );
15
16     shippingPaymentNotified = true;
17     emit Notify(
18         bank,
19         carrier,
20         "9. Banco notificou a transportadora sobre o pagamento do frete."
21     );
22 }
23
24 function deliverProduct() external onlyC {
25     require(productSent, "Produto ainda nao foi enviado pelo vendedor");
26
27     require(
28         shippingPaymentNotified,
29         "Regra Interna C: Transportadora so pode entregar apos NOTIFICACAO do banco."
30     );
31

```

```

32     state = ContractState.ProductDelivered;
33     emit Notify(
34         carrier,
35         buyer,
36         "10. Transportadora entregou o produto ao comprador."
37     );
38 }

```

Listagem 2 – Código solidity do contrato livre de conflito.

Os testes realizados com essa nova versão corrigida do contrato em Solidity na plataforma Remix IDE ocorreu da seguinte forma:

1. o contrato é criado com os respectivos endereços de cada participante, com um valor para o produto a venda (um inteiro qualquer) e valor associado ao frete;
2. a função *buyProduct* é executada pela conta do buyer, mudando o estado do contrato de *Created* para *ProductBought*;
3. após o compra do produto, de forma sequencial, a função *payProduct* também é executada pelo buyer e muda o estado do contrato de *Created* para *ProductPaid*;
4. na sequência, o banco notifica o pagamento através da função *notifyProductPayment*. Se a função é executada com sucesso, o controle do contrato passa para o estado *PaymentNotified*.
5. o seller então executa duas funções: *sendProduct* e *payShippingCosts*. Se as duas funções são executadas com sucesso, o estado do contrato é alterado para *PaymentNotified*;
6. em seguida, a função *deliverProduct*, é executada pela transportadora que verifica se o produto foi enviado e se os custos do frete foram pagos. Em caso afirmativo, o controle avança para *ProductDelivered* e um conflito não é detectado, pois as condições de execução são cumpridas.
7. na sequência, as funções *notifyProductReceipt* e *notifyProductDelivery* são executadas normalmente, pois o comprador notifica o recebimento do produto e na segunda, a transportadora notifica o envio do produto ao comprador.
8. por fim, as funções de pagamento *payProductSeller*, *liberateShippingCosts* e *payShippingCostsToCarrier* também rodaram sem a presença de erros. Na primeira, se o comprador recebe o produto então o pagamento do produto é liberado para o vendedor. Já na segunda, se a transportadora notifica a entrega então o pagamento do frete é liberado.

Ao final da execução do contrato, o vendedor recebe o valor do produto em sua conta com o valor do frete descontado. O comprador tem o valor do produto descontado de sua conta, e a transportadora recebe o valor do frete adicionado em sua conta.

5 CONCLUSÃO

REFERÊNCIAS

- [1] DUO, W.; XIN, H.; XIAOFENG, M. Formal analysis of smart contract based on colored petri nets. *IEEE Intelligent Systems*, v. 35, n. 3, p. 19–30, 2020.
- [2] AZZOPARDI, S.; COLOMBO, C.; PACE, G. Model-based static and runtime verification for ethereum smart contracts. p. 323–348, 02 2021.
- [3] MURA, W. A. D. Detecção de conflitos em contratos multilaterais. 2016.
- [4] PáDUA, F. Bizinoto Soares de. Contrato: O que É, suas funções e como entendê-lo. *Rev. Fac. Direito São Bernardo do Campo*, v. 26, n. 2. Disponível em: <<https://revistas.direitosbc.br/fdsbc/article/view/1011>>.
- [5] MIRANDA, M. B. Teoria geral dos contratos. *Revista Virtual Direito Brasil*, v. 2, n. 2, 2008. Disponível em: <<https://irp-cdn.multiscreensite.com/951f8786/files/uploaded/v22art3a.pdf>>.
- [6] VENOSA, S. de S. *Direito Civil, Teoria geral das obrigações e teoria geral dos contratos*. 4. ed. [S.l.]: atlas, 2004.
- [7] AGARWAL, S. E-contracts in practice: Case studies and legal precedents for effective implementation. *RESEARCH HUB International Multidisciplinary Research Journal*, v. 11, n. 1, 2024.
- [8] PRISACARIU, C.; SCHNEIDER, G. A formal language for electronic contracts*. *Formal Methods for Open Object-Based Distributed Systems*, 2007.
- [9] GOMES, N. G. Um panorama da lógica deôntica. *Revista de Filosofia*, 2008.
- [10] FENECH, S.; PACE, G. J.; SCHNEIDER, G. Conflict analysis of deontic contract. *WICT PROCEEDINGS*, 2008.
- [11] HERRESTAD, H.; KROGH, C. Deontic logic relativised to bearers and counterparties. *Proceedings of the Anniversary of Anthology in Computers and Law*, 1995.
- [12] BONIFACIO, A. L.; MURA, W. A. D. Automatically running experiments on checking multi-party contracts. *Artificial Intelligence and Law*, v. 29, n. 3, 2021.
- [13] NAKAMOTO, S. Bitcoin: A peer-to-peer electronic cash system. 2008.
- [14] MICHAELNOFER. Blockchain. *Business and Information Systems Engineering*, v. 59, n. 3, 2017.
- [15] SOLIDITY. *Solidity*. [S.l.], 2023. Disponível em: <<https://docs.soliditylang.org/en/latest/>>.
- [16] GUPTA, S. Blockchain transaction processing. *Encyclopedia of Big Data Technologies*, v. 1, n. 1, 2019.
- [17] PIERRO, M. D. What is blockchain? *IEEE*, v. 1, n. 1, 2017.

- [18] SZABO, N. Smart contracts: Building blocks for digital market. *EXTROPY: The Journal of Transhumanist Thought*, v. 16, 1996.
- [19] KõLVART, M.; POOLA, M.; RULL, A. Smart contracts. *The Future of Law and eTechnologies*, p. 133–147, 02 2016.
- [20] FILIPPI, P. D.; WRAY, C.; SILENO, G. Smart contracts. *Internet Policy Review*, v. 10, 2021.
- [21] TIKHOMIROV, S. Ethereum: State of knowledge and research perspectives. *Lecture Notes in Computer Science*, v. 10723, n. 1, 2017.
- [22] HILDENBRANDT, E. Kevm: A complete formal semantics of the ethereum virtual machine. *IEEE 31st Computer Security Foundations Symposium*, v. 1, n. 1, 2018.
- [23] WOOD, G.; REITWIESSNER, C. *Solidity, Vision and Roadmap*. [S.l.], 2014. Disponível em: <<https://archive.devcon.org/archive/watch/0/solidity-vision-and-roadmap/?tab=YouTube>>.
- [24] WOHRER, M.; ZDUN, U. Smart contracts: security patterns in the ethereum ecosystem and solidity. *IEEE*, 2018.
- [25] REMIX. *Remix*. [S.l.], 2025. Disponível em: <<https://remix-ide.readthedocs.io/en/latest/>>.
- [26] PETRI, C. A. *Kommunikation mit Automaten*. Tese (Doutorado) — Darmstadt University of Technology, Germany, 1962.
- [27] MACIEL, P. R. M.; LINS, R. D.; CUNHA, P. R. F. Introdução às redes de petri e aplicações. p. 201, 1996.
- [28] PETERSON, J. L. Petri nets. *ACM Computing Surveys (CSUR)*, ACM, v. 9, n. 3, p. 223–252, 1977.
- [29] LIU, F. *Colored Petri Nets for Systems Biology*. Tese (Doutorado) — Brandenburg University of Technology Cottbus, 2012.
- [30] KASSUYA, D. Y. F. Transformação entre contratos inteligentes e redes de petri. 11 2023.
- [31] CPN Tools. *Publications*. 2018. Acessado em: 12 de abril de 2026. Disponível em: <<https://cpntools.org/2018/01/16/publications/>>.
- [32] JENSEN, K.; KRISTENSEN, L. M. *Coloured Petri Nets: Modelling and Validation of Concurrent Systems*. Berlin, Heidelberg: Springer-Verlag Berlin Heidelberg, 2009. ISBN 978-3-642-00283-0.
- [33] Free Software Foundation. *GNU General Public License*. 2007. Disponível em: <<https://www.gnu.org/licenses/gpl-3.0.html>>. Acesso em: 13 de julho de 2026. Disponível em: <<https://www.gnu.org/licenses/gpl-3.0.html>>.